

Predictive analysis of naval incidents in the USA, 2002 - 2015:

Annex 5.1. Data Model VesselBalancedSample

Author: [Oscar Anton](#)

Date: 2024

License: [CC BY-NC-ND 4.0 DEED](#)

Version: 0.9

0. Loadings

Libraries

```
In [1]: # System environment
import os

# Data general management
import numpy as np
import pandas as pd

# Visualization
import matplotlib.pyplot as plt
import graphviz
import seaborn as sns

# Model training
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, KFold, cross_val_score, ShuffleSplit
from sklearn.ensemble import BaggingClassifier, GradientBoostingClassifier, RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.tree import DecisionTreeClassifier, export_graphviz
from sklearn.linear_model import LogisticRegression
from xgboost import XGBClassifier

# Model metrics
from sklearn.metrics import roc_auc_score, accuracy_score, log_loss, cohen_kappa_score,
from sklearn.inspection import permutation_importance
from scipy.stats import ttest_rel

# Model Export
import joblib

# Model explainers
import dalex as dx
```

General variables

```
In [2]: # Main data folders
merged_activity_folder = '../3.DataPreprocess/DataMergedActivity'
```

```

datasets_folder = 'Datasets'
models_folder = 'Models'

# Toggle for export data to external file
file_export_enabled = False
# Toggle for train model
train_model_enabled = False

# Available CPU cores for multiprocessing (training models)
n_jobs = os.cpu_count() - 1
# Random seed for reproducibility
seed = 42

```

Base dataframe

```

In [3]: # Load dataframe
VesselBalancedSample = pd.read_feather(merged_activity_folder + '/' + 'VesselBalancedSam

# Check dataframe
VesselBalancedSample.head(5)

```

```

Out[3]:

```

	vessel_id	imo_number	vessel_name	vessel_class	build_year	gross_ton	length	flag_abbr	cl
0	5820		ISABELLA MARIE	Recreational	1999	14	32.8	US	
1	170582		TERMINATOR	Fishing Vessel	1979	17	38.1	US	
2	257931		SUMMER ISLE	Recreational	1984	8	29.9	US	
3	151752		NORJERNAN	Passenger Ship	1976	18	35.2	US	
4	308953		NONSENSE	Recreational	1987	8	27.0	US	

1. Dataframe creation

1.1. Variable transformation

Note: Missing and NA values previously processed. Data balanced by design

1.1.1. Category variables: Reduce excessive variability

```

In [4]: # Function: cutting values
def cut_years(column):
    return pd.cut(column,
                    bins=[-float('inf'), 1940, 1960, 1980, 2000, float('inf')],
                    labels=['very Old', 'old', 'average', 'new', 'very new'],
                    include_lowest=True)

# Function: group minority values
def lump_factorials(column, prop=0.008, other_level="other value"):
    counts = column.value_counts(normalize=True)

```

```

mask = column.isin(counts[counts < prop].index)
column[mask] = other_level
return column

# First transformations: assign 'y': No event = 0, otherwise = 1
# Cutting, renaming, dropping not relevant variables
data_general = (
    VesselBalancedSample
    .assign(y=lambda x: pd.factorize(x['event_type'] != "No event", sort=True)[0])
    .assign(build_year=cut_years(VesselBalancedSample['build_year']))
    .rename(columns={'length': 'vessel_length'})
    .drop(columns=['vessel_id', 'imo_number', 'vessel_name', 'event_type', 'damage_statu
)

# Group minority values
data_general[['vessel_class', 'flag_abbr', 'classification_society', 'solas_desc']] = (d
    .apply(lump_factorials)
)

# Check structure
data_general.dtypes

```

```

Out[4]: vessel_class      object
build_year      category
gross_ton       int32
vessel_length   float64
flag_abbr       object
classification_society  object
solas_desc      object
y              int64
dtype: object

```

1.1.2. Category variables: one-hot-encoding

```

In [5]: # One hot encoding for not numeric variables
data_ohe = (pd.get_dummies(data_general
    .select_dtypes(exclude=['number']))
    .astype(int))

```

1.1.3. Numeric variables: Scale

```

In [6]: # Apply standard scaler
data_scaled = pd.DataFrame(StandardScaler()
    .fit_transform(data_general[['gross_ton', 'vessel_length']]))

# Rename column names as strings
data_scaled.columns = ['gross_ton', 'vessel_length']

```

1.2. Join data

```

In [7]: # Dataset join: encoded, scaled and 'y'
data_num = pd.concat([data_ohe,
    data_scaled,
    data_general['y']],
    axis=1)

# Rename column names as strings
data_num.columns = data_num.columns.astype(str)

```

```
# Verify variables
for column in data_num.columns:
    print(f"Name: {column} | Type: {data_num[column].dtype} | Levels: {data_num[column].
```

```
Name: vessel_class_Barge | Type: int32 | Levels: 2
Name: vessel_class_Bulk Carrier | Type: int32 | Levels: 2
Name: vessel_class_Fishing Vessel | Type: int32 | Levels: 2
Name: vessel_class_General Dry Cargo Ship | Type: int32 | Levels: 2
Name: vessel_class_Miscellaneous Vessel | Type: int32 | Levels: 2
Name: vessel_class_Offshore | Type: int32 | Levels: 2
Name: vessel_class_Passenger Ship | Type: int32 | Levels: 2
Name: vessel_class_Recreational | Type: int32 | Levels: 2
Name: vessel_class_Tank Ship | Type: int32 | Levels: 2
Name: vessel_class_Towing Vessel | Type: int32 | Levels: 2
Name: vessel_class_other value | Type: int32 | Levels: 2
Name: build_year_very Old | Type: int32 | Levels: 2
Name: build_year_old | Type: int32 | Levels: 2
Name: build_year_average | Type: int32 | Levels: 2
Name: build_year_new | Type: int32 | Levels: 2
Name: build_year_very new | Type: int32 | Levels: 2
Name: flag_abbr_CA | Type: int32 | Levels: 2
Name: flag_abbr_LR | Type: int32 | Levels: 2
Name: flag_abbr_PA | Type: int32 | Levels: 2
Name: flag_abbr_US | Type: int32 | Levels: 2
Name: flag_abbr_other value | Type: int32 | Levels: 2
Name: classification_society_AMERICAN BUREAU OF SHIPPING | Type: int32 | Levels: 2
Name: classification_society_DET NORSKE VERITAS | Type: int32 | Levels: 2
Name: classification_society_LLOYD'S REGISTER OF SHIPPING | Type: int32 | Levels: 2
Name: classification_society_NIPPON KAIJI KYOKAI | Type: int32 | Levels: 2
Name: classification_society_UNSPECIFIED | Type: int32 | Levels: 2
Name: classification_society_other value | Type: int32 | Levels: 2
Name: solas_desc_Active SOLAS | Type: int32 | Levels: 2
Name: solas_desc_Historical SOLAS | Type: int32 | Levels: 2
Name: solas_desc_Non SOLAS | Type: int32 | Levels: 2
Name: gross_ton | Type: float64 | Levels: 6622
Name: vessel_length | Type: float64 | Levels: 4597
Name: y | Type: int64 | Levels: 2
```

```
In [8]: # Save dataset
if file_export_enabled :
    data_num.reset_index().to_feather(datasets_folder + '/' + 'data_num.feather')
    print(f'data_num {data_num.shape} exported to {datasets_folder}')
else:
    data_num = pd.read_feather(datasets_folder + '/' + 'data_num.feather')
    print(f'data_num {data_num.shape} imported from {datasets_folder}')
```

data_num (109836, 34) imported from Datasets

1.3. Train / Test Split

```
In [9]: # Data split
X_train, X_test, y_train, y_test = train_test_split(
    data_num.drop(columns=['y']),
    data_num['y'],
    test_size=0.2,
    random_state=42)
```

```
In [10]: # Save / Load dataframes in one file (h5 format for multiple data)
if file_export_enabled :
    dfs = {'X_train':X_train, 'X_test':X_test, 'y_train':y_train, 'y_test':y_test}
    for key, df in dfs.items():
        df.to_hdf(datasets_folder + '/' + 'datasets_split.h5', key=key, format='table')
```

```

        print(f'{key} {eval(key).shape} exported to {datasets_folder} folder')
    else:
        dfs = ['X_train', 'X_test', 'y_train', 'y_test']
        for df in dfs:
            globals()[df] = pd.read_hdf(datasets_folder + '/' + 'datasets_splited.h5', key =
            print(f'{df} {eval(df).shape} imported from {datasets_folder} folder')

```

X_train (87868, 32) imported from Datasets folder

X_test (21968, 32) imported from Datasets folder

y_train (87868,) imported from Datasets folder

y_test (21968,) imported from Datasets folder

2. Model Training

Performance displaying functions

Parameters

```

In [11]: # Function: Show Parameters of trained model
def show_parameters(model):
    # Get model parameters and print as a one row list
    print(f'Parameters:')
    params = model.get_params()
    for param_name, param_value in params.items():
        print(f" {param_name}: {param_value}")

```

Metrics

```

In [12]: # Function: Table with main metrics data
def model_metrics(model, X, y, styled=False):
    # Predictions (absolute)
    y_pred = model.predict(X)

    # Calculate main metrics
    roc_auc = round(roc_auc_score(y, y_pred), 4)
    accuracy = round(accuracy_score(y, y_pred), 4)
    loss = round(log_loss(y, y_pred), 4)
    kappa = round(cohen_kappa_score(y, y_pred), 4)
    mcc = round(matthews_corrcoef(y, y_pred), 4)
    f1 = round(f1_score(y, y_pred), 4)

    # Sensitivity And Specificity
    tn, fp, fn, tp = confusion_matrix(y, y_pred).ravel()
    sensitivity = round(tp / (tp + fn), 4)
    specificity = round(tn / (tn + fp), 4)

    # Build multiindex table
    df = pd.DataFrame([['ROC AUC:', roc_auc], ['Accuracy:', accuracy], ['Log loss:', loss],
                      ['Kappa:', kappa], ['MCC:', mcc], ['F1:', f1], [' ', ''],
                      ['Sensitivity:', sensitivity], ['Specificity:', specificity]],
                      columns=('metric', 'value'))

    if styled:
        title = f'{model.__class__.__name__} Training'
        df.columns = pd.MultiIndex.from_tuples([(title, col) for col in df.columns])
        return df.style.hide()
    else:
        return df

```

Confusion Matrix

```
In [13]: # Function: Table with Confusion Matrix data
def confusion_matrix_table(model, X, y):
    # Predictions (absolute)
    y_pred = model.predict(X)

    # Confusion matrix
    tn, fp, fn, tp = confusion_matrix(y, y_pred).ravel()

    # Dataframe creation
    df = pd.DataFrame([[tp, fn], [fp, tn]],
                      index=pd.Index(['1', '0'], name='Actual Label:'),
                      columns=pd.MultiIndex.from_product([model.__class__.__name__], ['1', '0']))

    # Dataframe style
    styled_df = df.style.set_table_styles([
        {'selector': 'th.col_heading', 'props': 'text-align: center;'},
        {'selector': 'td', 'props': 'text-align: center;'},
    ], overwrite=False)

    return styled_df
```

ROC curve

```
In [14]: # Function: Plot ROC Curve
def plot_roc_curve(model, X, y):
    # Predicted probabilities
    y_score = model.predict_proba(X)

    # Calculate ROC for each class
    fpr, tpr, _ = roc_curve(y, y_score[:, 1])

    # Calculate AUC (Area Under Curve)
    roc_auc = auc(fpr, tpr)

    # Plot ROC Curve
    plt.figure()
    plt.plot(fpr, tpr, color='darkorange', lw=2, label='ROC curve (AUC = %0.2f)' % roc_auc)
    plt.plot([0, 1], [0, 1], color='black', lw=1, linestyle='dotted')
    plt.xlim([0, 1])
    plt.ylim([0, 1.01])
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title(f'ROC Curve for {model.__class__.__name__}')
    plt.legend(loc="lower right")
    plt.show()
```

Feature importances

```
In [15]: # Function: Built a dataframe for feature importances (for sklearn regular models)
def sklearn_importances(model):
    # Dataframe creation for importance values in sklearn, sorted by importance_value
    importances = (pd.DataFrame({'variable_name': model.feature_names_in_,
                                'importance_value': model.feature_importances_})
                  .sort_values(by='importance_value', ascending=False)
                  .reset_index(drop=True))

    # Add custom attributes for model name and calculation method
    importances.model_name = model.__class__.__name__
    importances.importance_method = 'feature_importances_ (sklearn)'
```

```
return importances
```

```
In [16]: # Function: Built a dataframe for feature importances, according to feature permutation
def permutation_importances(model, X, y, repeats=3):
    # Feature permutation importance calculation (from sklearn.inspection)
    importance_values = permutation_importance(model, X, y, n_repeats=repeats, random_state=None)

    # Importance absolute values and normalization
    importance_values = np.array(abs(importance_values))
    importance_values = importance_values / importance_values.sum()

    # Built importances dataframe
    importances = (pd.DataFrame({'variable_name': X.columns,
                                'importance_value': importance_values})
                   .sort_values(by='importance_value', ascending=False)
                   .reset_index(drop=True))

    # Add custom attribute for model name and calculation method
    importances.model_name = model.__class__.__name__
    importances.importance_method = 'feature permutation'

    return importances
```

```
In [17]: # Function: Horizontal bar plot for importance values
def plot_importances(importances_df, top=25):
    plt.figure()
    bars = plt.barh(importances_df['variable_name'][:top], importances_df['importance_value'][:top])
    plt.title(f'Top {top} Feature importances of {importances_df.model_name} model')
    plt.xlabel(f'Relative Feature importance, according to {importances_df.importance_method}')
    plt.gca().invert_yaxis()

    # Add value labels on the bars
    for i, bar in enumerate(bars):
        plt.text(0, bar.get_y() + bar.get_height() / 2, f'{importances_df["importance_value"][i]:.2f}')

    plt.show()
```

2.1. Bayesian networks

2.1.1. Naïve Bayes -Gaussian- (NB)

Model Training

```
In [18]: if train_model_enabled :
    # Define model parameters
    params = {} # No parameters for this model

    # Create and train model
    nb_train = GaussianNB(**params).fit(X_train, y_train)

    # Save to external file
    joblib.dump(nb_train, models_folder + '/' + 'nb_train.pkl')
    print(f'{nb_train.__class__.__name__} model exported to {models_folder} folder')
else:
    # Load model from external file
    nb_train = joblib.load(models_folder + '/' + 'nb_train.pkl')
    print(f'{nb_train.__class__.__name__} model loaded from {models_folder} folder')

# Call previous function to show used parameters
```

```
show_parameters(nb_train)
```

GaussianNB model loaded from Models folder

Parameters:

priors: None

var_smoothing: 1e-09

Model Main Metrics

```
In [19]: # Call function to show main metrics
model_metrics(nb_train, X_test, y_test, styled=True)
```

Out[19]: **GaussianNB Training**

metric	value
ROC AUC:	0.673700
Accuracy:	0.674900
Log loss:	11.716500
Kappa:	0.348200
MCC:	0.388200
F1:	0.580000

Sensitivity: 0.451400

Specificity: 0.895900

```
In [20]: # Call function to show confusion matrix
confusion_matrix_table(nb_train, X_test, y_test)
```

Out[20]: **Model: GaussianNB**

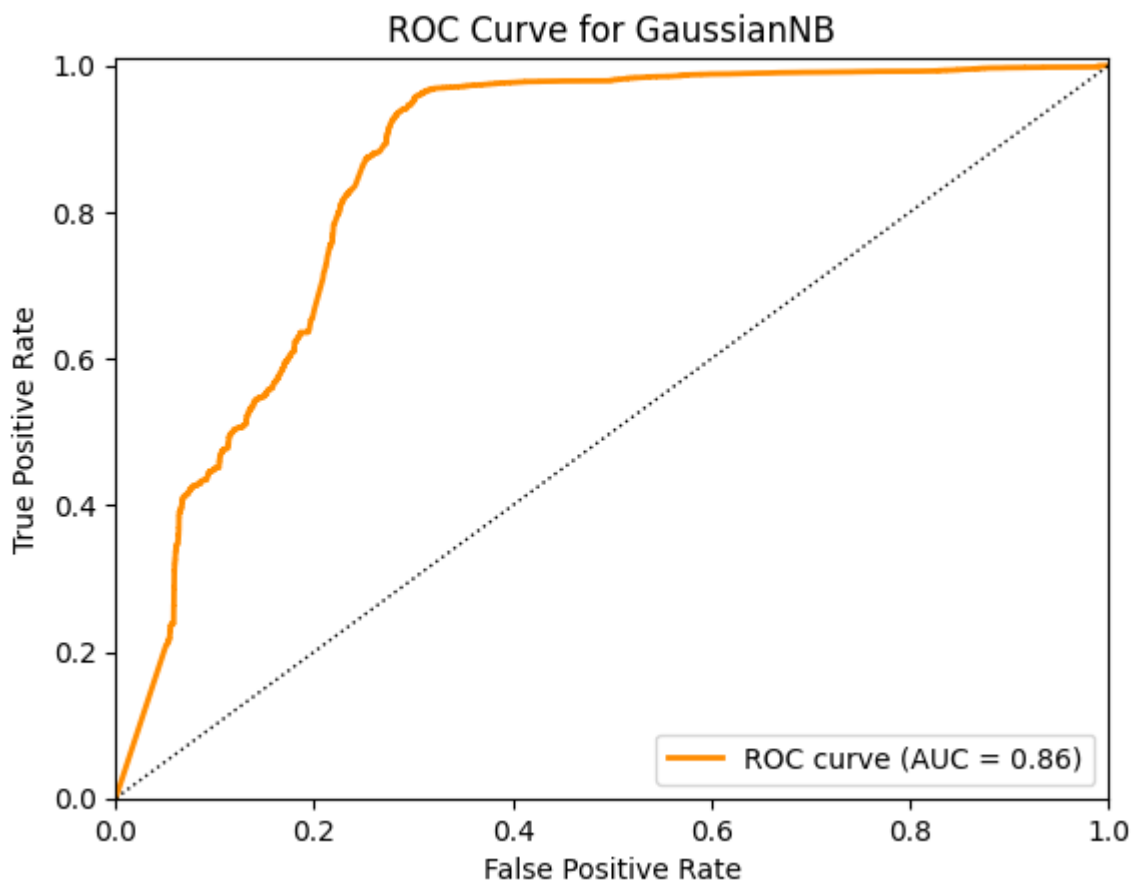
Predicted: 1 0

Actual Label:

1 4930 5991

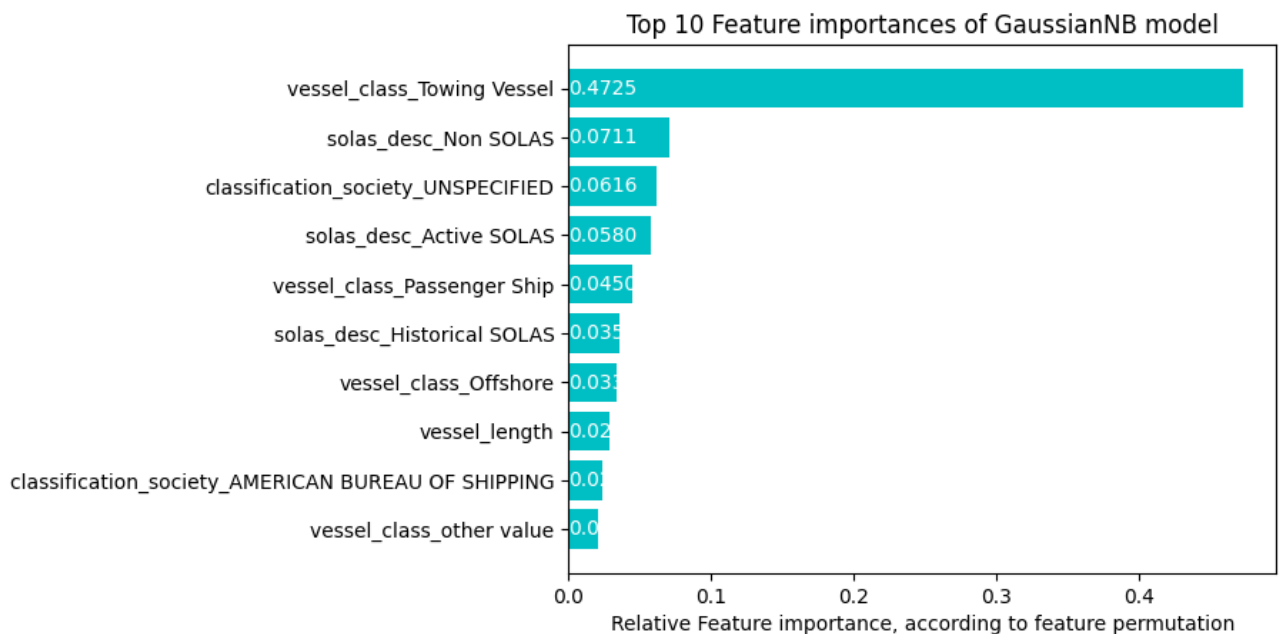
0 1150 9897

```
In [21]: # Call function to plot Receiver Operating Characteristic (ROC) curve
plot_roc_curve(nb_train, X_test, y_test)
```



Feature importances (permutation)

In [22]: `# Call previous functions for plotting top 10 importances according to theta values`
`plot_importances(permutation_importances(nb_train, X_train, y_train), top=10)`



2.1.2. Bagged Naïve Bayes -Gaussian- (BNB)

Model Training

In [23]: `if train_model_enabled :`
 `# Define model parameters`
 `params = {}` `# No parameters for this model`
`# Create and train model.`

```

bnb_train = BaggingClassifier(GaussianNB(**params), n_estimators=5).fit(X_train, y_train)

# Save to external file
joblib.dump(bnb_train, models_folder + '/' + 'bnb_train.pkl')
print(f'{bnb_train.__class__.__name__} model exported to {models_folder} folder')
else:
    # Load model from external file
    bnb_train = joblib.load(models_folder + '/' + 'bnb_train.pkl')
    print(f'{bnb_train.__class__.__name__} model loaded from {models_folder} folder')

# Bagging (Bootstrap Aggregating) is a technique used to improve the stability and accuracy
# n_estimators specifies how many individual Naive Bayes classifiers are trained independently
# It approximates the behavior of AODE

# Call previous function to show used parameters
show_parameters(bnb_train)

```

GaussianNB model loaded from Models folder

Parameters:

```

bootstrap: True
bootstrap_features: False
estimator__priors: None
estimator__var_smoothing: 1e-09
estimator: GaussianNB()
max_features: 1.0
max_samples: 1.0
n_estimators: 5
n_jobs: None
oob_score: False
random_state: None
verbose: 0
warm_start: False

```

Model Main Metrics

```

In [24]: # Call function to show main metrics
model_metrics(bnb_train, X_test, y_test, styled=True)

```

Out[24]: **BaggingClassifier Training**

metric	value
ROC AUC:	0.673800
Accuracy:	0.675100
Log loss:	11.709900
Kappa:	0.348600
MCC:	0.388800
F1:	0.579900

Sensitivity:	0.451000
Specificity:	0.896700

```

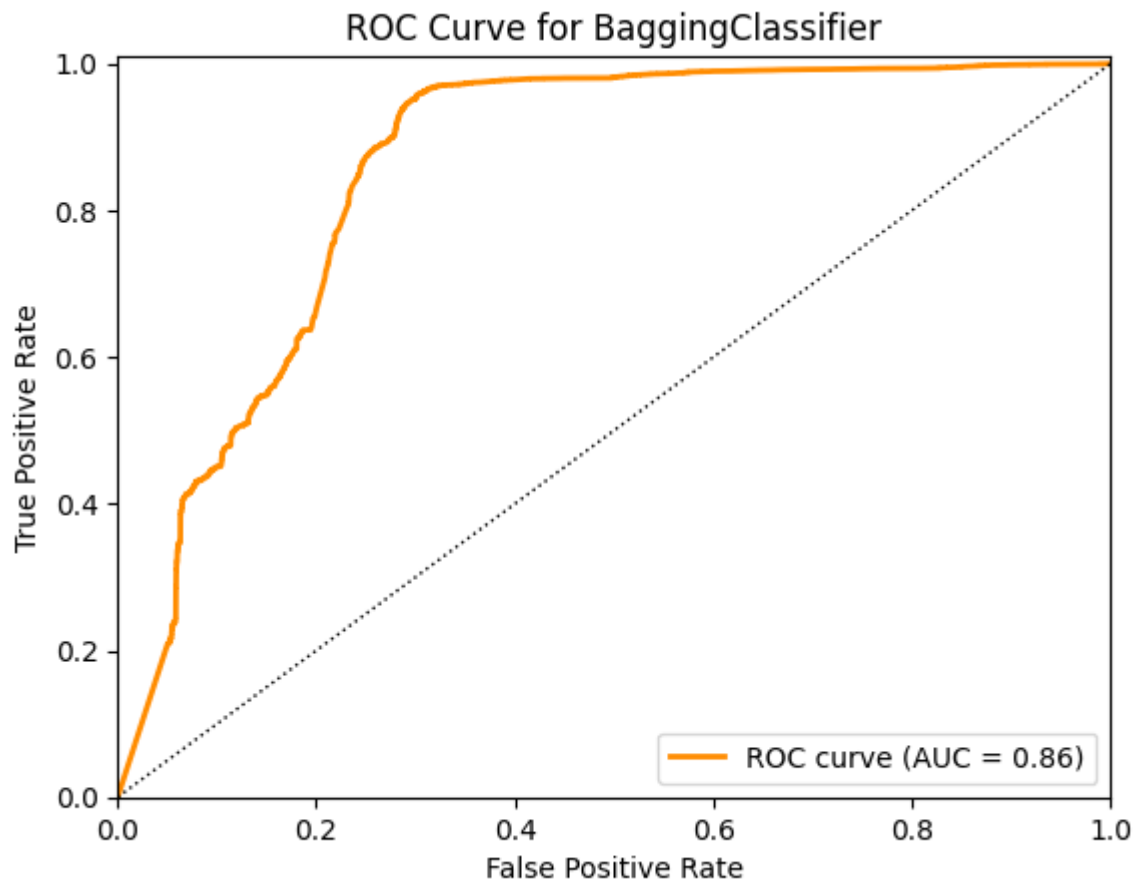
In [25]: # Call function to show confusion matrix
confusion_matrix_table(bnb_train, X_test, y_test)

```

Out[25]:

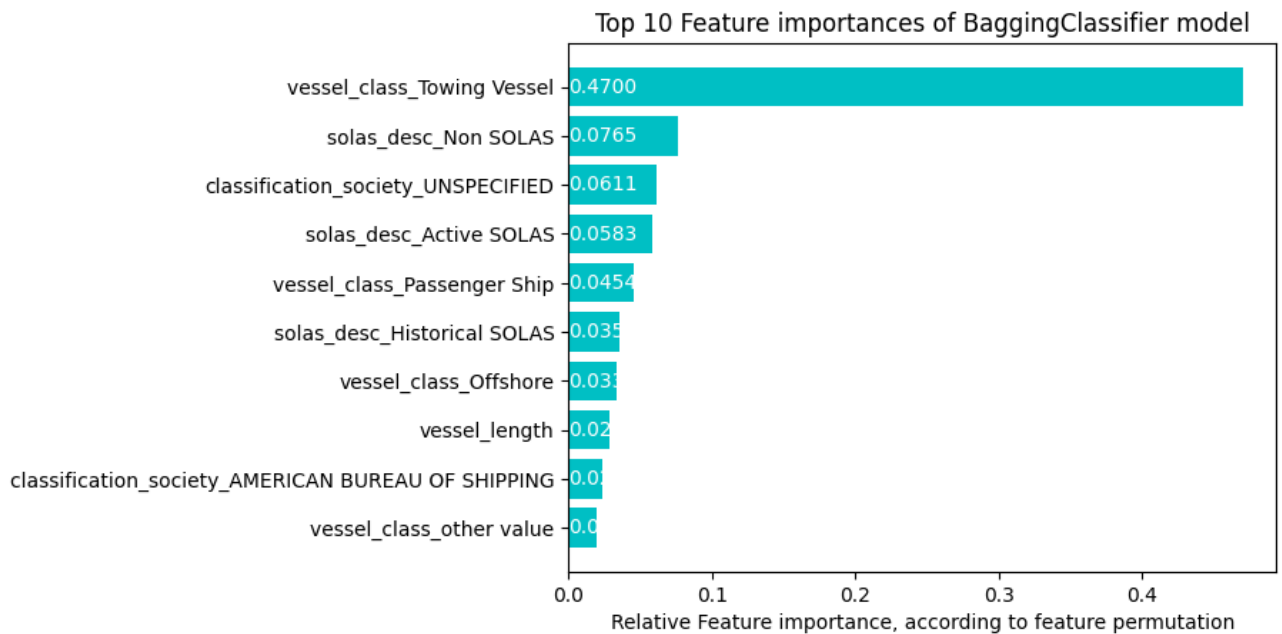
Model: BaggingClassifier**Predicted: 1 0****Actual Label:****1** 4925 5996**0** 1141 9906

In [26]: *# Call function to plot Receiver Operating Characteristic (ROC) curve*
`plot_roc_curve(bnb_train, X_test, y_test)`



Feature importances (permutation)

In [27]: *# Call previous functions for plotting top 10 importances according to feature permutation*
`plot_importances(permutation_importances(bnb_train, X_train, y_train), top=10)`



2.2. Gradient Boosting Models

2.2.1. Gradient Boosting Machine (GBM)

Model Training

```
In [28]: if train_model_enabled :
# Define model parameters
params = {
    'n_estimators': 500,           # Number of trees
    'learning_rate': 0.1,         # Contribution of each tree to the model
    'max_depth': 3,               # Maximum levels of each tree
    'random_state': seed,         # Random seed for reproducibility
}

# Create and train model
GBM_train = GradientBoostingClassifier(**params).fit(X_train, y_train)

# Save to external file
joblib.dump(GBM_train, models_folder + '/' + 'GBM_train.pkl')
print(f'{GBM_train.__class__.__name__} model exported to {models_folder} folder')
else:
# Load model from external file
GBM_train = joblib.load(models_folder + '/' + 'GBM_train.pkl')
print(f'{GBM_train.__class__.__name__} model loaded from {models_folder} folder')

# Call previous function to show used parameters
show_parameters(GBM_train)
```

GradientBoostingClassifier model loaded from Models folder

Parameters:

```
ccp_alpha: 0.0
criterion: friedman_mse
init: None
learning_rate: 0.1
loss: log_loss
max_depth: 3
max_features: None
max_leaf_nodes: None
min_impurity_decrease: 0.0
min_samples_leaf: 1
min_samples_split: 2
min_weight_fraction_leaf: 0.0
n_estimators: 500
n_iter_no_change: None
random_state: 42
subsample: 1.0
tol: 0.0001
validation_fraction: 0.1
verbose: 0
warm_start: False
```

Model Main Metrics

```
In [29]: # Call function to show main metrics
model_metrics(GBM_train, X_test, y_test, styled=True)
```

Out[29]: **GradientBoostingClassifier Training**

metric	value
ROC AUC:	0.865100
Accuracy:	0.864800
Log loss:	4.871300
Kappa:	0.729900
MCC:	0.734000
F1:	0.871000
Sensitivity:	0.917600
Specificity:	0.812700

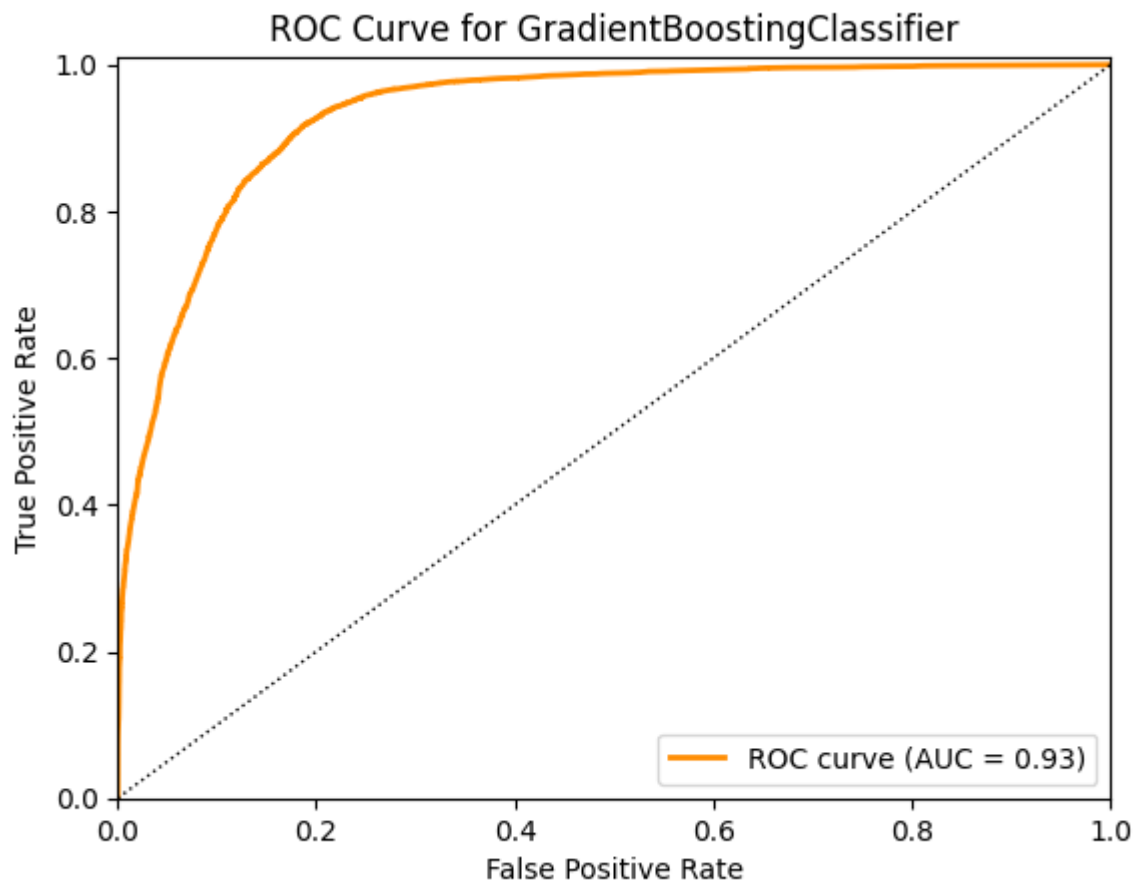
```
In [30]: # Call function to show confusion matrix
confusion_matrix_table(GBM_train, X_test, y_test)
```

Out[30]: **Model: GradientBoostingClassifier**

Predicted:	1	0
Actual Label:		
1	10021	900
0	2069	8978

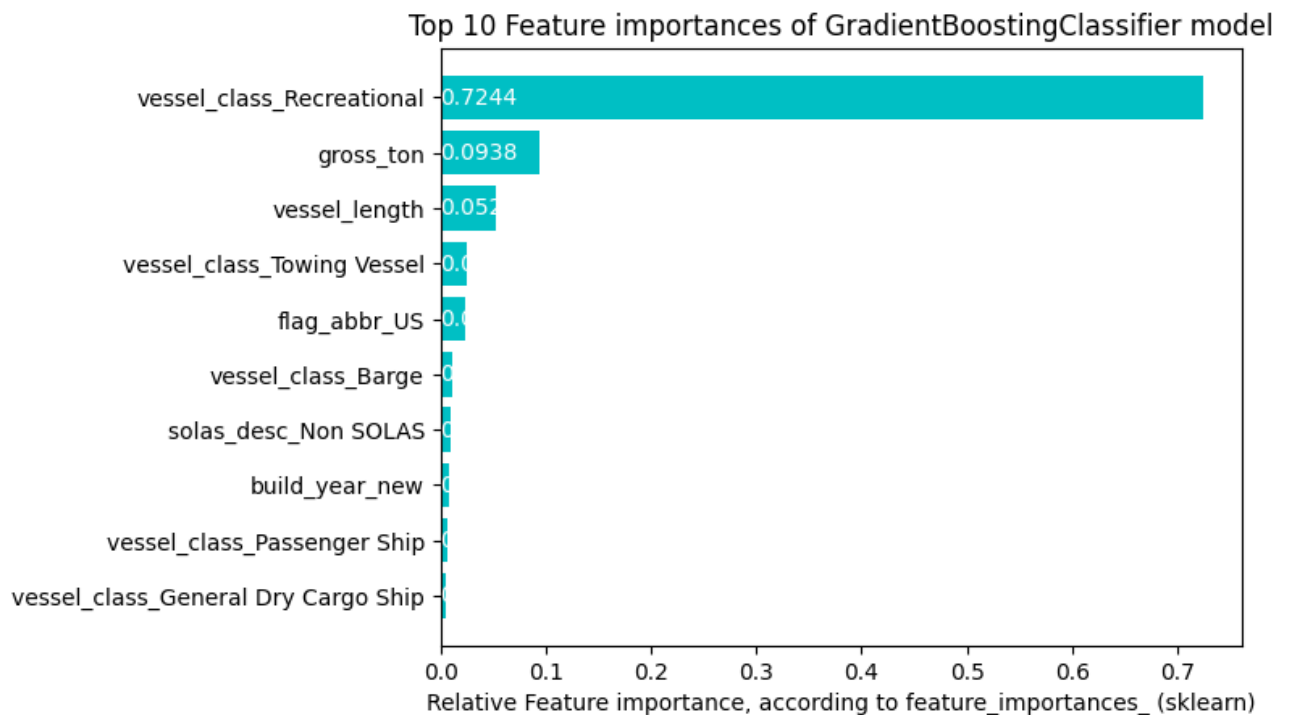
```
In [31]: # Call function to plot Receiver Operating Characteristic (ROC) curve
```

```
plot_roc_curve(GBM_train, X_test, y_test)
```



Feature importance

```
In [32]: # Call previous functions for plotting top 10 importances according to feature_importances_
plot_importances(sklearn_importances(GBM_train), top=10)
```



Extra: Main metrics per iteration (n_estimators=500)

```
In [33]: # Number of iterations during training
iterations = int(GBM_train.get_params().get("n_estimators") + 1)
```

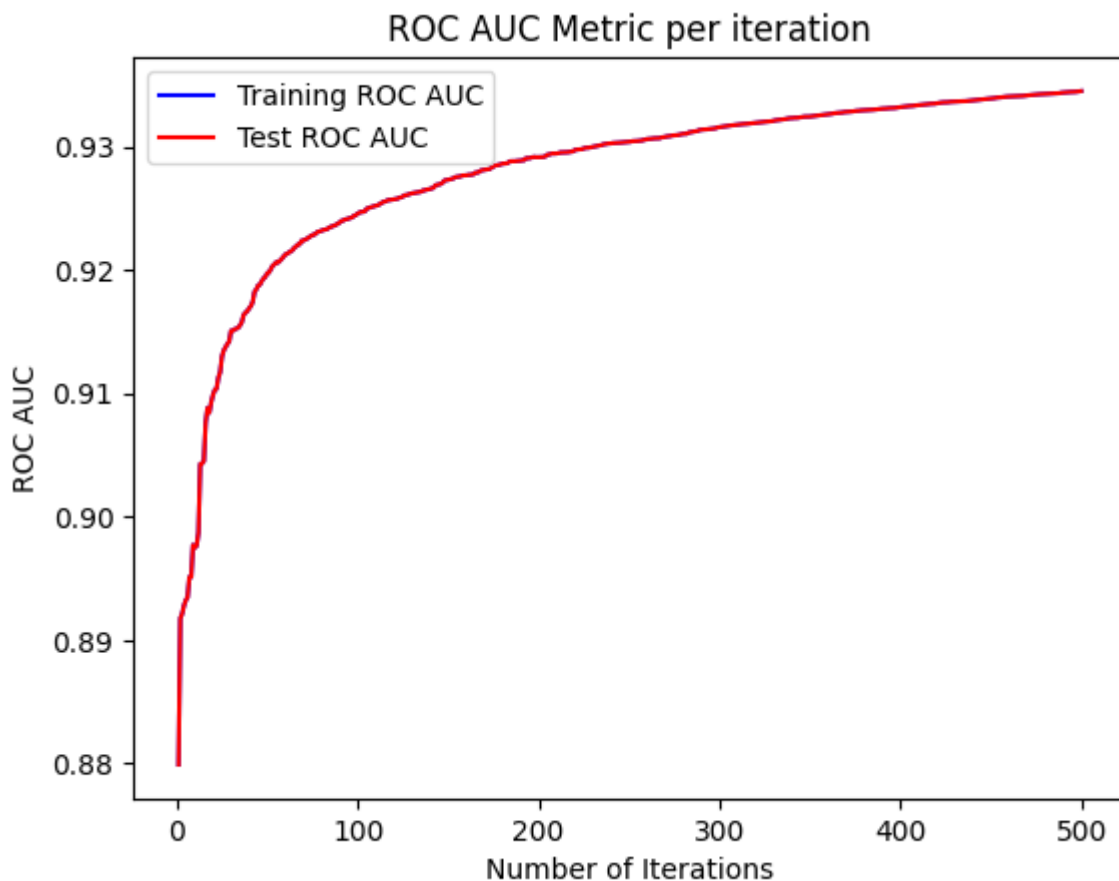
```

# Obtain prediction probabilities at each stage of training
train_probs = np.array([preds[:, 1] for preds in GBM_train.staged_predict_proba(X_train)])
test_probs = np.array([preds[:, 1] for preds in GBM_train.staged_predict_proba(X_test)])

# Calculate ROC metric at each stage of training
train_roc_auc = [roc_auc_score(y_train, prob) for prob in train_probs]
test_roc_auc = [roc_auc_score(y_train, prob) for prob in train_probs]

# Plot the ROC metric as a function of iterations
plt.plot(range(1, iterations), train_roc_auc, label="Training ROC AUC", color="blue")
plt.plot(range(1, iterations), test_roc_auc, label="Test ROC AUC", color="red")
plt.xlabel("Number of Iterations")
plt.ylabel("ROC AUC")
plt.title("ROC AUC Metric per iteration")
plt.legend()
plt.show()

```

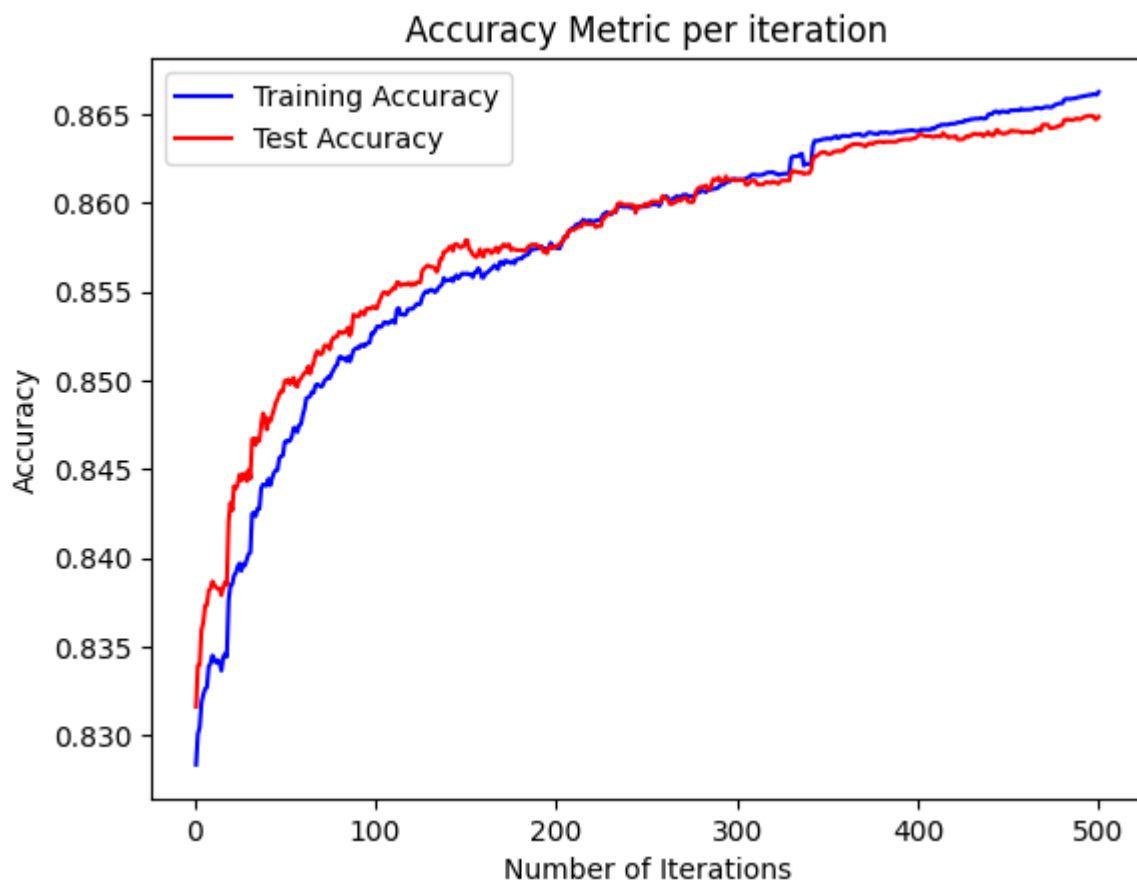


```

In [34]: # Get predictions at each stage of training
train_errors = [accuracy_score(y_train, y_pred) for y_pred in GBM_train.staged_predict(X_train)]
test_errors = [accuracy_score(y_test, y_pred) for y_pred in GBM_train.staged_predict(X_test)]

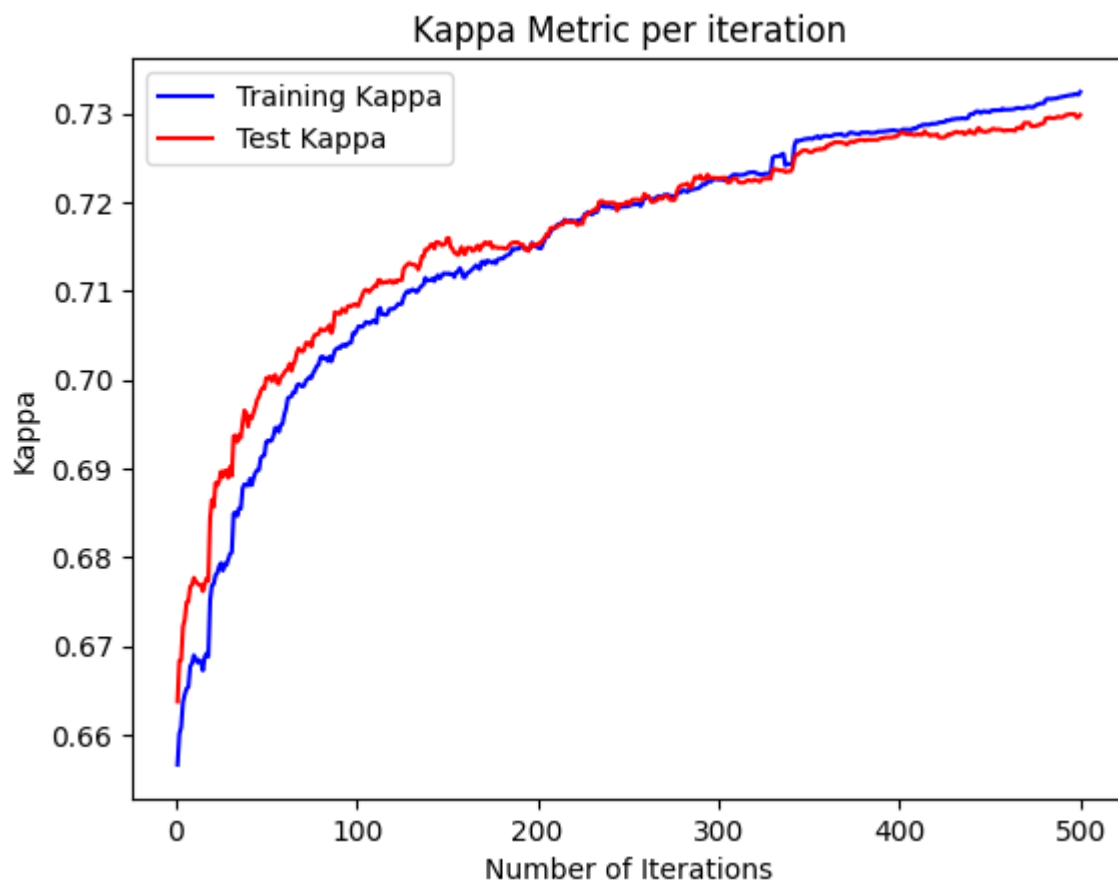
# Plot the Accuracy metric as a function of iterations
plt.plot(np.arange(1, iterations), train_errors, label="Training Accuracy", color="blue")
plt.plot(np.arange(1, iterations), test_errors, label="Test Accuracy", color="red")
plt.xlabel("Number of Iterations")
plt.ylabel("Accuracy")
plt.title("Accuracy Metric per iteration")
plt.legend()
plt.show()

```



```
In [35]: # Get predictions at each stage of training
train_kappa = [cohen_kappa_score(y_train, y_pred) for y_pred in GBM_train.staged_predict(X_train)]
test_kappa = [cohen_kappa_score(y_test, y_pred) for y_pred in GBM_train.staged_predict(X_test)]

# Plot the Kappa metric as a function of iterations
plt.plot(np.arange(1, iterations), train_kappa, label="Training Kappa", color="blue")
plt.plot(np.arange(1, iterations), test_kappa, label="Test Kappa", color="red")
plt.xlabel("Number of Iterations")
plt.ylabel("Kappa")
plt.title("Kappa Metric per iteration")
plt.legend()
plt.show()
```

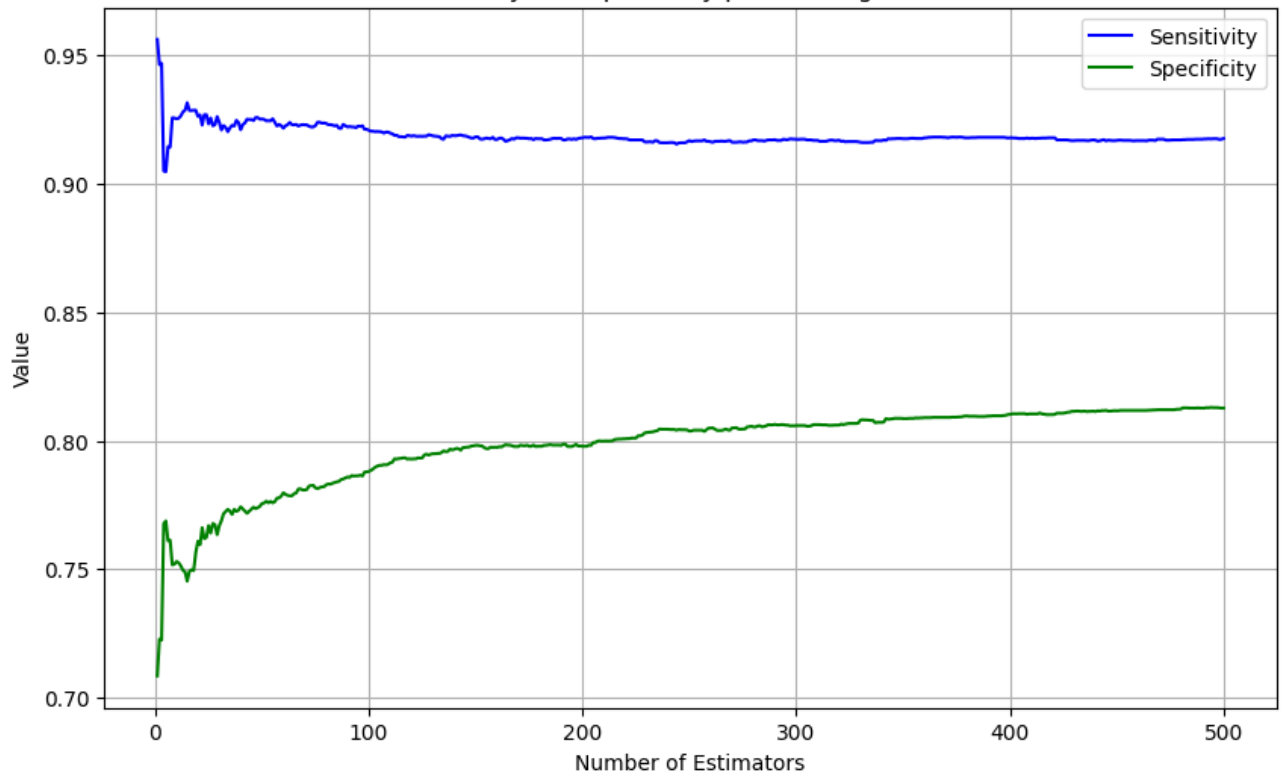


```
In [36]: sensitivities = []
         specificities = []

         # Get values per each iteration
         for i, y_pred in enumerate(GBM_train.staged_predict(X_test)):
             tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
             sensitivity = tp / (tp + fn)
             specificity = tn / (tn + fp)
             sensitivities.append(sensitivity)
             specificities.append(specificity)

         # Plot sensitivity and specificity for each iteration of training
         plt.figure(figsize=(10, 6))
         plt.plot(range(1, iterations), sensitivities, label='Sensitivity', color='blue')
         plt.plot(range(1, iterations), specificities, label='Specificity', color='green')
         plt.xlabel('Number of Estimators')
         plt.ylabel('Value')
         plt.title('Sensitivity and Specificity per Training Iteration')
         plt.legend()
         plt.grid(True)
         plt.show()
```

Sensitivity and Specificity per Training Iteration



2.2.2. Extreme Gradient Boosting (XGB)

Model Training

```
In [37]: if train_model_enabled :
# Define model parameters
params = {
    'objective': 'binary:logistic',      # Binary classification problem
    'eval_metric': 'error',              # Evaluation metric: error rate
    'eta': 0.1,                          # Learning rate
    'max_depth': 3,                      # Maximum tree depth
    'colsample_bytree': 0.3,              # Subsample ratio of columns when constructing e
    'random_state': seed                 # Random seed for reproducibility
}

# Create and train model
XGB_train = XGBClassifier(**params).fit(X_train, y_train)

# Save to external file
joblib.dump(XGB_train, models_folder + '/' + 'XGB_train.pkl')
print(f'{XGB_train.__class__.__name__} model exported to {models_folder} folder')
else:
# Load model from external file
XGB_train = joblib.load(models_folder + '/' + 'XGB_train.pkl')
print(f'{XGB_train.__class__.__name__} model loaded from {models_folder} folder')

# Call previous function to show used parameters
show_parameters(XGB_train)
```

XGBClassifier model loaded from Models folder

Parameters:

```
objective: binary:logistic
base_score: None
booster: None
callbacks: None
colsample_bylevel: None
colsample_bynode: None
colsample_bytree: 0.3
device: None
early_stopping_rounds: None
enable_categorical: False
eval_metric: error
feature_types: None
gamma: None
grow_policy: None
importance_type: None
interaction_constraints: None
learning_rate: None
max_bin: None
max_cat_threshold: None
max_cat_to_onehot: None
max_delta_step: None
max_depth: 3
max_leaves: None
min_child_weight: None
missing: nan
monotone_constraints: None
multi_strategy: None
n_estimators: None
n_jobs: None
num_parallel_tree: None
random_state: 42
reg_alpha: None
reg_lambda: None
sampling_method: None
scale_pos_weight: None
subsample: None
tree_method: None
validate_parameters: None
verbosity: None
eta: 0.1
```

Model Main Metrics

```
In [38]: # Call function to show main metrics
model_metrics(XGB_train, X_test, y_test, styled=True)
```

Out[38]: **XGBClassifier Training**

metric	value
ROC AUC:	0.851500
Accuracy:	0.851100
Log loss:	5.366800
Kappa:	0.702400
MCC:	0.708800
F1:	0.859700

Sensitivity: 0.917500

Specificity: 0.785500

In [39]: *# Call function to show confusion matrix*
 confusion_matrix_table(XGB_train, X_test, y_test)

Out[39]: **Model: XGBClassifier**

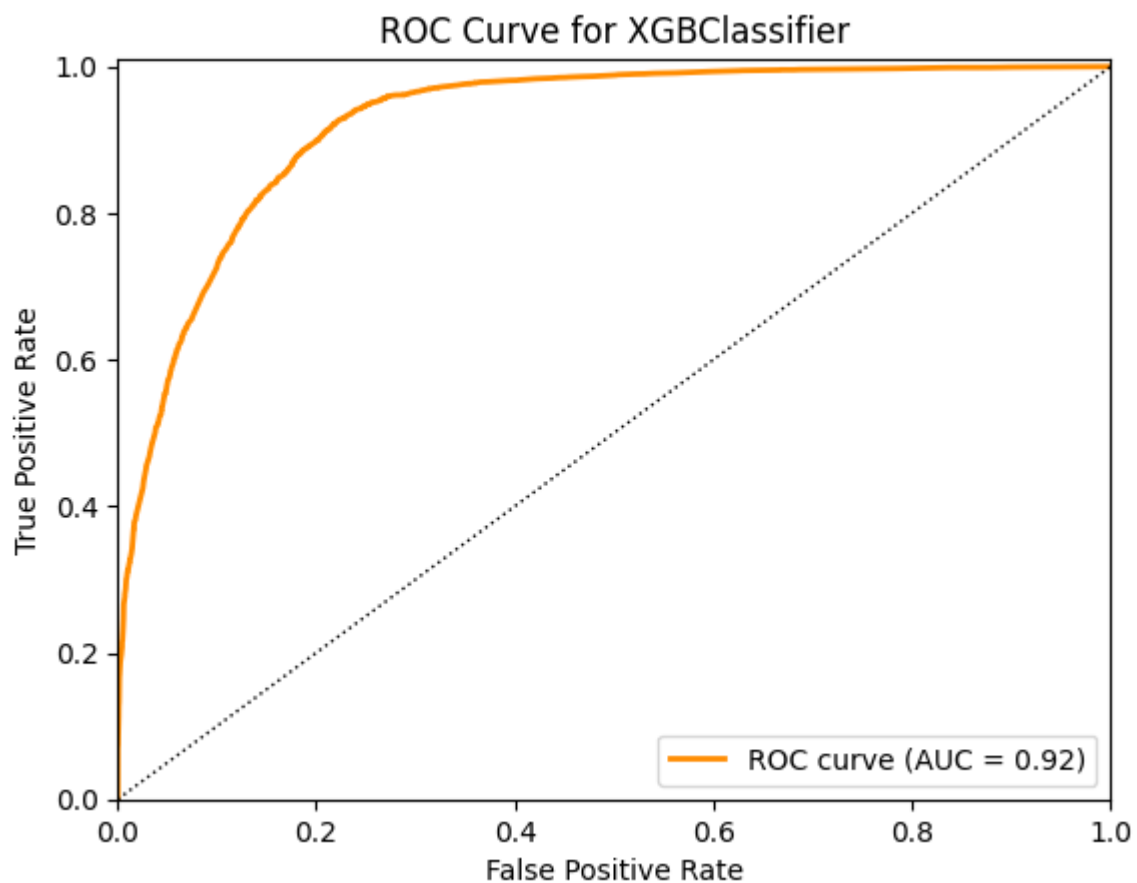
Predicted: 1 0

Actual Label:

1 10020 901

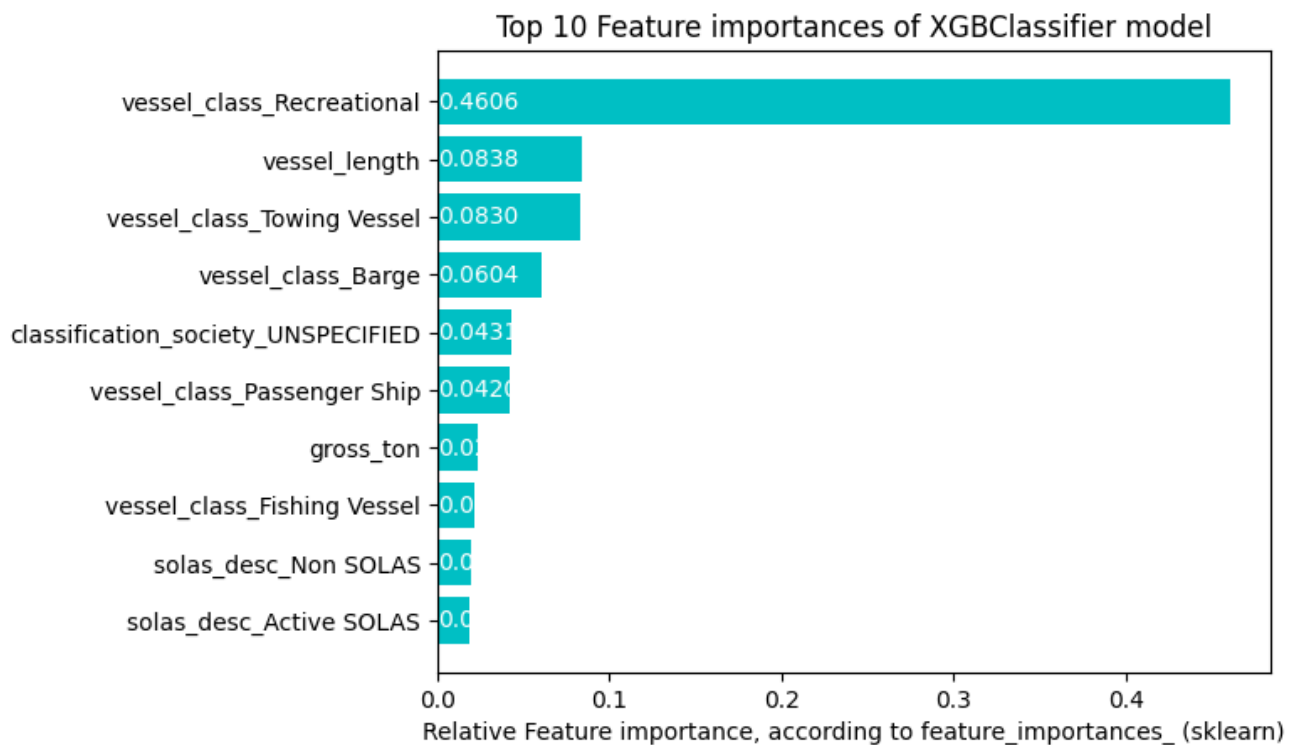
0 2370 8677

In [40]: *# Call function to plot Receiver Operating Characteristic (ROC) curve*
 plot_roc_curve(XGB_train, X_test, y_test)



Feature importance

```
In [41]: # Call previous functions for plotting top 10 importances according to feature_importance
plot_importances(sklearn_importances(XGB_train), top=10)
```



2.3. More models

2.3.1. Random Forest (RF)

Model Training

```
In [42]: if train_model_enabled :
# Define model parameters
params = {
    'n_estimators': 100,          # Number of trees in the forest
    'max_depth': None,           # Maximum depth of the trees (no restrictions)
    'random_state': seed        # Random seed for reproducibility
}

# Create and train model
rf_train = RandomForestClassifier(**params).fit(X_train, y_train)

# Save to external file
joblib.dump(rf_train, models_folder + '/' + 'rf_train.pkl')
print(f'{rf_train.__class__.__name__} model exported to {models_folder} folder')
else:
# Load model from external file
rf_train = joblib.load(models_folder + '/' + 'rf_train.pkl')
print(f'{rf_train.__class__.__name__} model loaded from {models_folder} folder')

# Call previous function to show used parameters
show_parameters(rf_train)
```

RandomForestClassifier model loaded from Models folder

Parameters:

```
bootstrap: True
ccp_alpha: 0.0
class_weight: None
criterion: gini
max_depth: None
max_features: sqrt
max_leaf_nodes: None
max_samples: None
min_impurity_decrease: 0.0
min_samples_leaf: 1
min_samples_split: 2
min_weight_fraction_leaf: 0.0
monotonic_cst: None
n_estimators: 100
n_jobs: None
oob_score: False
random_state: 42
verbose: 0
warm_start: False
```

Model Main Metrics

```
In [43]: # Call function to show main metrics
model_metrics(rf_train, X_test, y_test, styled=True)
```

Out[43]: **RandomForestClassifier Training**

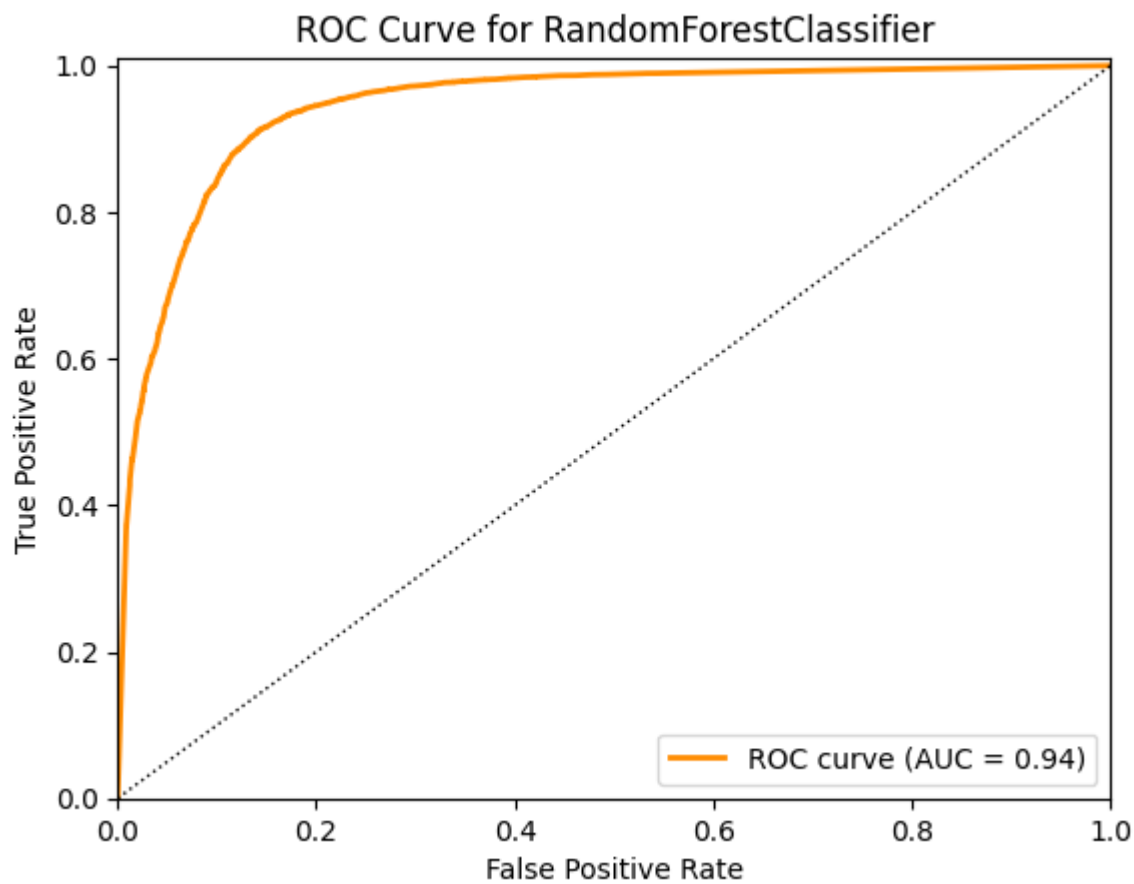
metric	value
ROC AUC:	0.883000
Accuracy:	0.882800
Log loss:	4.224900
Kappa:	0.765700
MCC:	0.768000
F1:	0.886600
Sensitivity:	0.921300
Specificity:	0.844700

```
In [44]: # Call function to show confusion matrix
confusion_matrix_table(rf_train, X_test, y_test)
```

Out[44]: **Model: RandomForestClassifier**

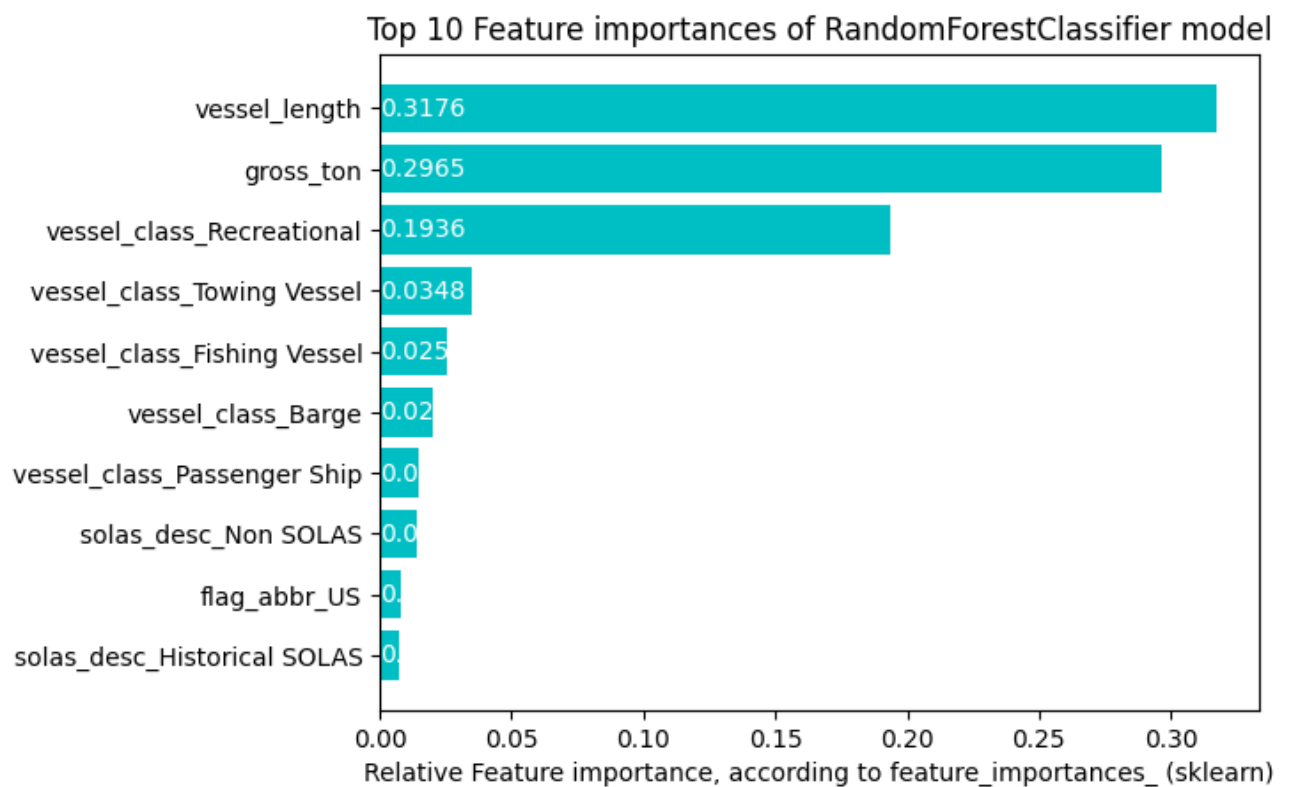
Predicted:	1	0
Actual Label:		
1	10062	859
0	1716	9331

```
In [45]: # Call function to plot Receiver Operating Characteristic (ROC) curve
plot_roc_curve(rf_train, X_test, y_test)
```



Feature importance

```
In [46]: # Call previous functions for plotting top 10 importances according to feature_importances_
plot_importances(sklearn_importances(rf_train), top=10)
```



Extra: Metrics per iteration (n_estimators=100)

```
In [47]: # Define the number of iterations, initial number of estimators, and step size
n_iterations = 10
initial_n_estimators = 10
```

```

step_size = 10

# Create empty lists to store the number of estimators and ROC AUC scores
num_estimators_list = []
roc_auc_scores = []

# Iterate over the specified number of iterations
for i in range(n_iterations):
    # Create the Random Forest model with the current number of estimators
    rf_model = RandomForestClassifier(n_estimators=initial_n_estimators + i * step_size,

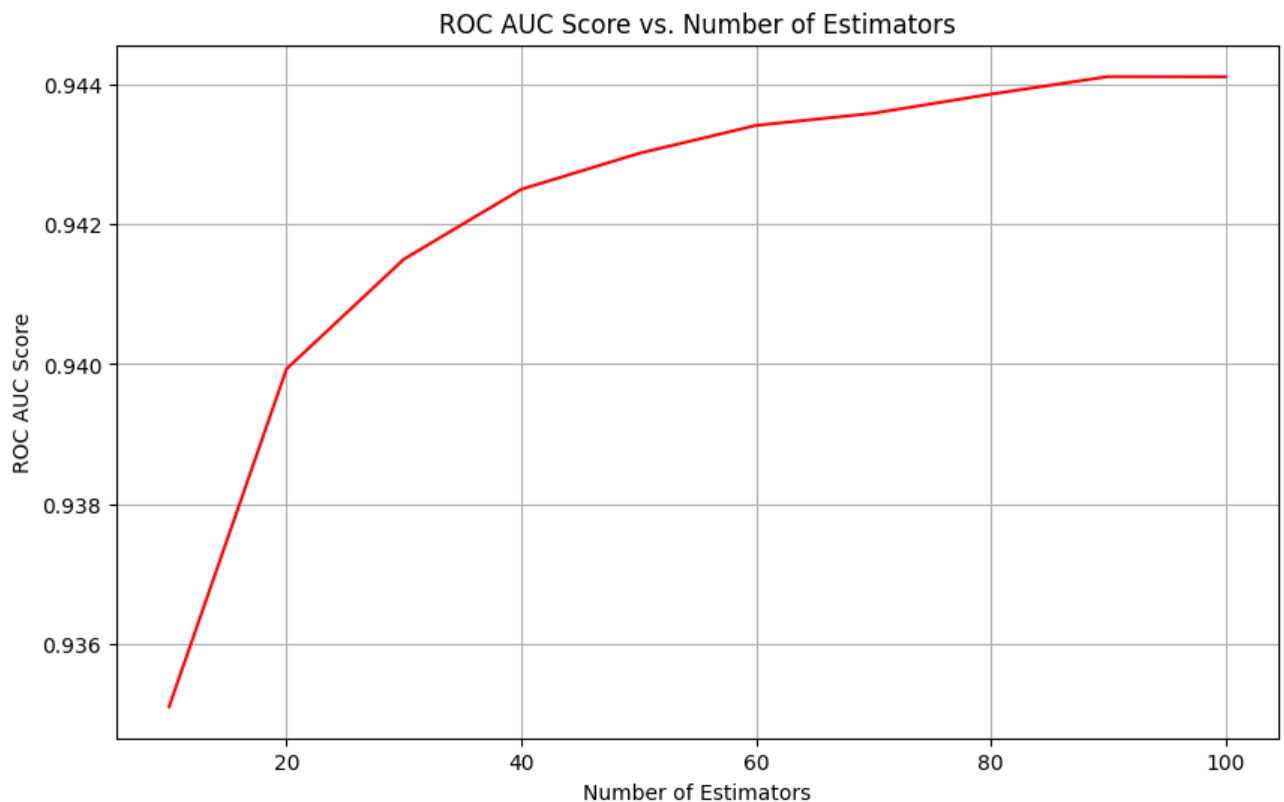
    # Train the model
    rf_model.fit(X_train, y_train)

    # Make predictions and calculate ROC AUC score
    y_pred_proba = rf_model.predict_proba(X_test)[: , 1]
    roc_auc = roc_auc_score(y_test, y_pred_proba)

    # Store the number of estimators and ROC AUC score
    num_estimators_list.append(initial_n_estimators + i * step_size)
    roc_auc_scores.append(roc_auc)

# Plot the ROC AUC scores over the number of estimators
plt.figure(figsize=(10, 6))
plt.plot(num_estimators_list, roc_auc_scores, label="ROC AUC", color="red")
plt.title('ROC AUC Score vs. Number of Estimators')
plt.xlabel('Number of Estimators')
plt.ylabel('ROC AUC Score')
plt.grid(True)

```



```

In [48]: # Create empty lists to store the number of estimators and ROC AUC scores
num_estimators_list = []
accuracy_auc_scores = []

# Iterate over the specified number of iterations
for i in range(n_iterations):
    # Create the Random Forest model with the current number of estimators
    rf_model = RandomForestClassifier(n_estimators=initial_n_estimators + i * step_size,

```

```

# Train the model
rf_model.fit(X_train, y_train)

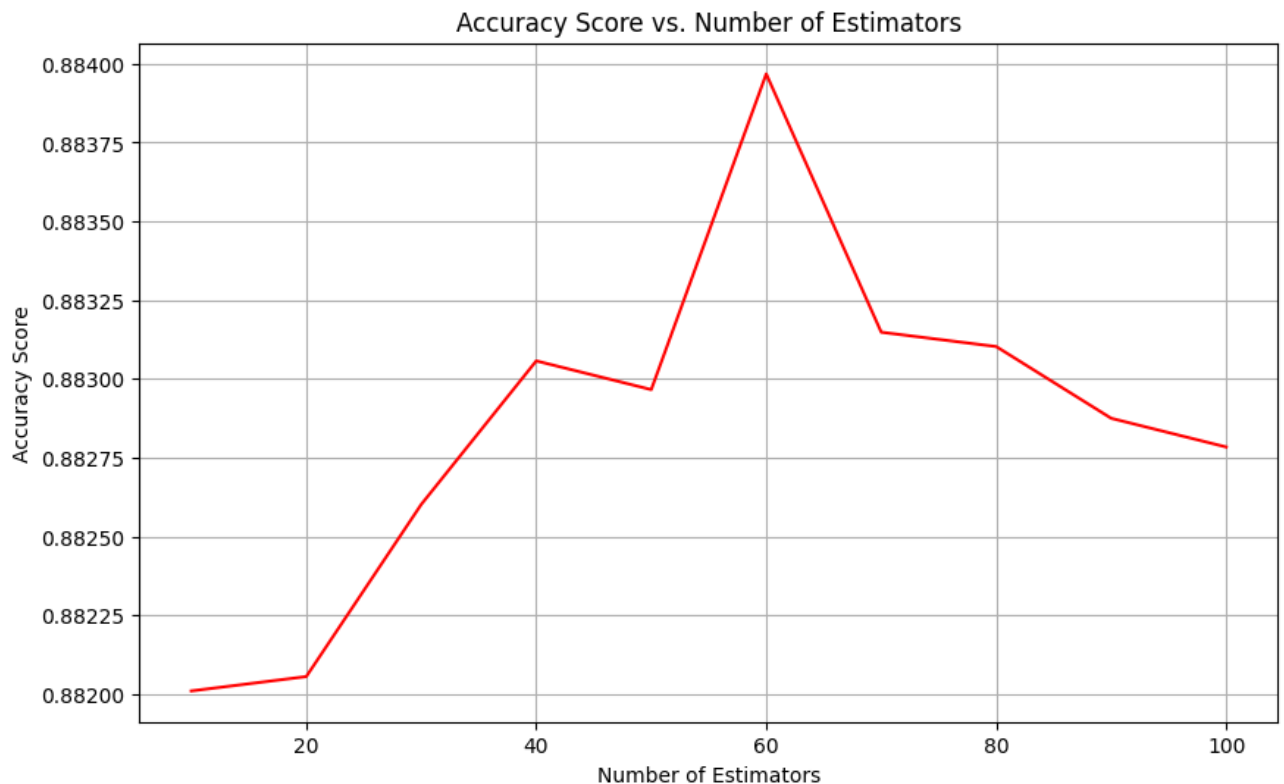
# Predictions
y_pred = rf_model.predict(X_test)

# Calculate Accuracy scores
accuracy_auc = accuracy_score(y_test, y_pred)

# Store the number of estimators and Accuracy score
num_estimators_list.append(initial_n_estimators + i * step_size)
accuracy_auc_scores.append(accuracy_auc)

# Plot the Accuracy scores over the number of estimators
plt.figure(figsize=(10, 6))
plt.plot(num_estimators_list, accuracy_auc_scores, label="Accuracy", color="red")
plt.title('Accuracy Score vs. Number of Estimators')
plt.xlabel('Number of Estimators')
plt.ylabel('Accuracy Score')
plt.grid(True)

```



```

In [49]: # Create empty Lists to store the number of estimators and ROC AUC scores
num_estimators_list = []
kappa_scores = []

# Iterate over the specified number of iterations
for i in range(n_iterations):
    # Create the Random Forest model with the current number of estimators
    rf_model = RandomForestClassifier(n_estimators=initial_n_estimators + i * step_size,

    # Train the model
    rf_model.fit(X_train, y_train)

    # Predictions
    y_pred = rf_model.predict(X_test)

    # Calculate Kappa scores
    kappa = cohen_kappa_score(y_test, y_pred)

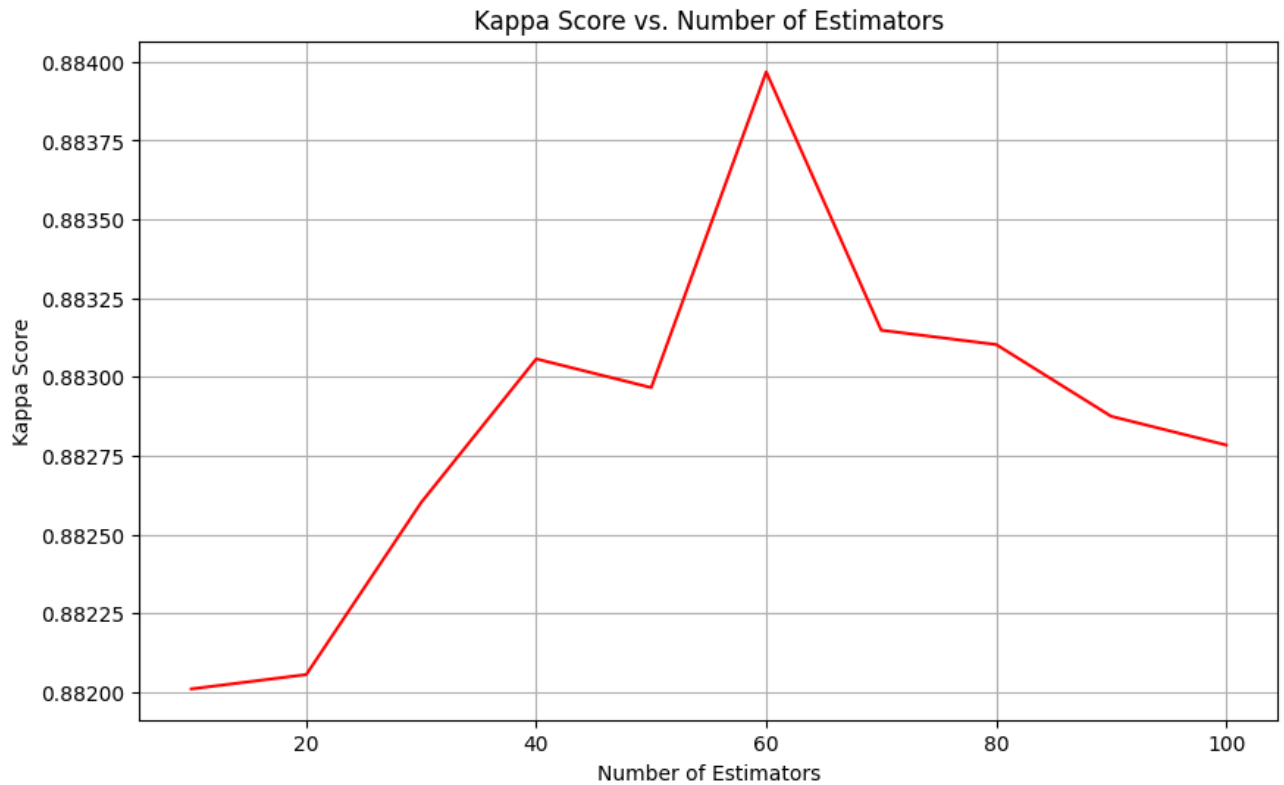
```

```

# Store the number of estimators and Kappa scores
num_estimators_list.append(initial_n_estimators + i * step_size)
kappa_scores.append(kappa)

# Plot the Kappa scores over the number of estimators
plt.figure(figsize=(10, 6))
plt.plot(num_estimators_list, accuracy_auc_scores, label="Kappa", color="red")
plt.title('Kappa Score vs. Number of Estimators')
plt.xlabel('Number of Estimators')
plt.ylabel('Kappa Score')
plt.grid(True)

```



Extra 2: Explore structure of a tree example

```

In [50]: if file_export_enabled :
# Extract a random model tree (for example, the first one)
tree = rf_train.estimators_[0]

# Export the tree to Graphviz
dot_data = export_graphviz(tree, out_file=None,
                           filled=True, rounded=True,
                           special_characters=True, precision=5)
graph = graphviz.Source(dot_data)

# Export to pdf file (lighter than svg)
graph.render('rf_train_tree', format='pdf', cleanup=True)

# Note: File is not displayed because is extremely width

```

2.3.2. Support Vector Machine -Radial Kernel- (SVM)

Model Training

```

In [51]: if train_model_enabled :
# Define model parameters
params = {

```

```

    'kernel': 'rbf',                # Radial Kernel
    'probability': True,            # Enable probability estimates (uses 5-fold cross-validation)
    'random_state': seed            # Random seed for reproducibility
}

# Create and train model
svm_train = SVC(**params).fit(X_train, y_train)

# Save to external file
joblib.dump(svm_train, models_folder + '/' + 'svm_train.pkl')
print(f'{svm_train.__class__.__name__} model exported to {models_folder} folder')
else:
    # Load model from external file
    svm_train = joblib.load(models_folder + '/' + 'svm_train.pkl')
    print(f'{svm_train.__class__.__name__} model loaded from {models_folder} folder')

# Call previous function to show used parameters
show_parameters(svm_train)

```

SVC model loaded from Models folder

Parameters:

```

C: 1.0
break_ties: False
cache_size: 200
class_weight: None
coef0: 0.0
decision_function_shape: ovr
degree: 3
gamma: scale
kernel: rbf
max_iter: -1
probability: True
random_state: 42
shrinking: True
tol: 0.001
verbose: False

```

Model Main Metrics

```

In [52]: # Call function to show main metrics
model_metrics(svm_train, X_test, y_test, styled=True)

```

Out[52]: **SVC Training**

metric	value
ROC AUC:	0.849200
Accuracy:	0.848700
Log loss:	5.453800
Kappa:	0.697700
MCC:	0.710100
F1:	0.860900

Sensitivity: 0.941600

Specificity: 0.756900

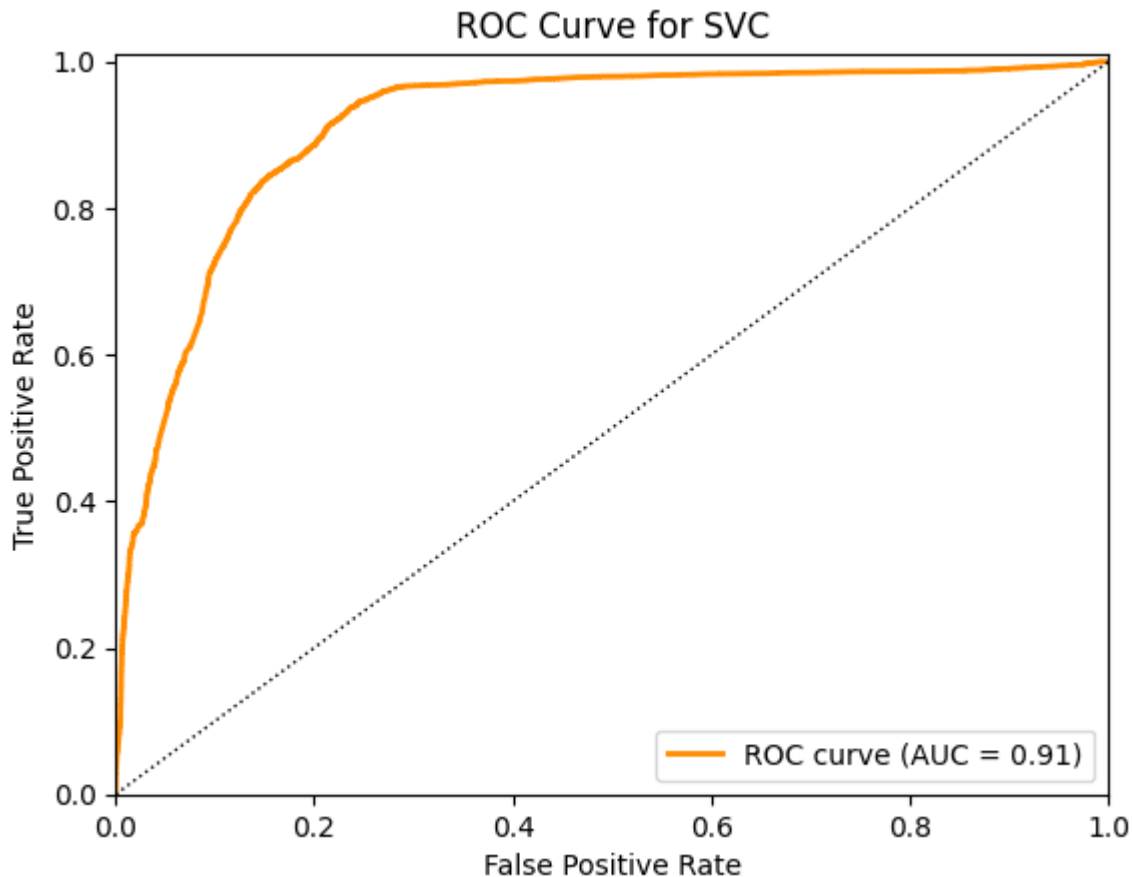
```
In [53]: # Call function to show confusion matrix
confusion_matrix_table(svm_train, X_test, y_test)
```

```
Out[53]:      Model:      SVC

      Predicted:      1      0

Actual Label:
-----
      1  10283   638
      0   2686  8361
```

```
In [54]: # Call function to plot Receiver Operating Characteristic (ROC) curve
plot_roc_curve(svm_train, X_test, y_test)
```

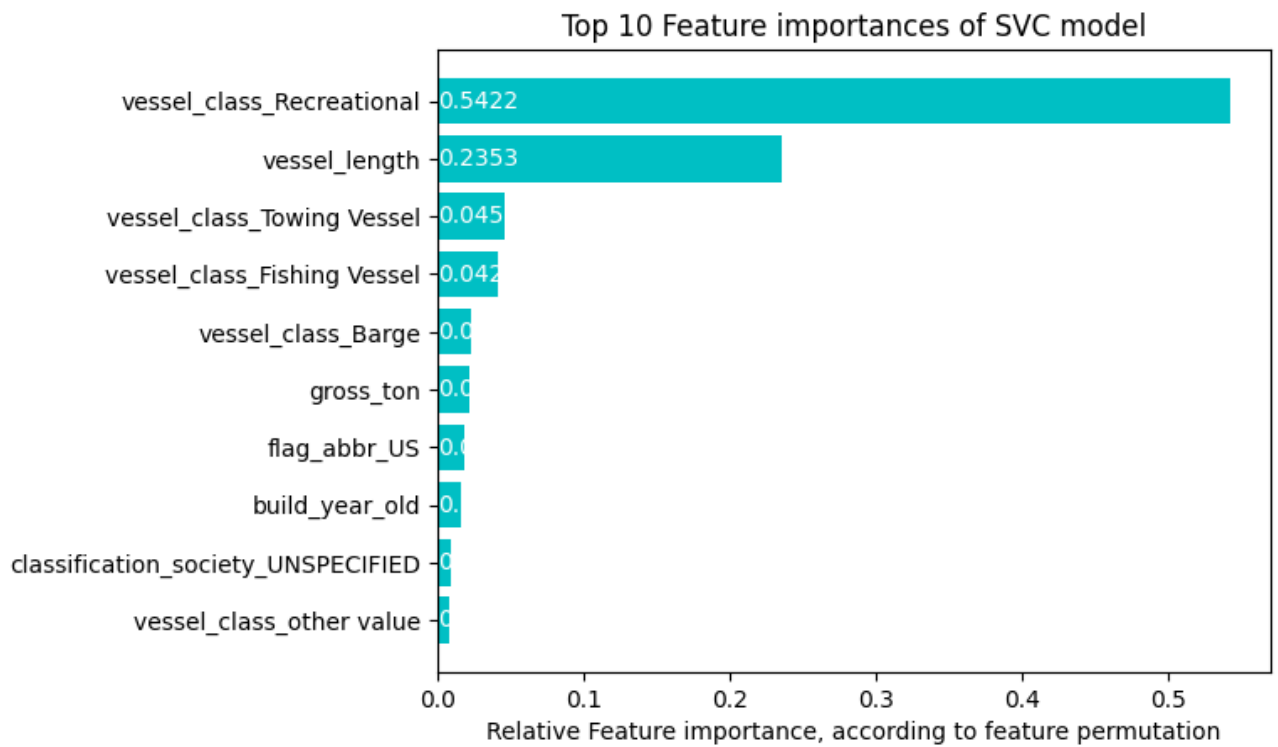


Feature importance (permutation)

```
In [55]: # Due to use of a non-linear kernel, coef_ or other direct methods are not available. Fe
# In this case, it's time-consuming calculation, therefore data will be exported for eas

if file_export_enabled :
    # Calculate feature permutation importances
    svm_importances = permutation_importances(svm_train, X_test, y_test, repeats=2)
    # Save to external file
    joblib.dump(svm_importances, models_folder + '/' + 'svm_importances.pkl')
else :
    # Load model from external file
    svm_importances = joblib.load(models_folder + '/' + 'svm_importances.pkl')
    svm_importances.model_name = svm_train.__class__.__name__
    svm_importances.importance_method = 'feature permutation'

# Call previous functions for plotting top 10 importances
plot_importances(svm_importances, top=10)
```



2.3.3. Multilayer Perceptron (MLP)

Model Training

```
In [56]: if train_model_enabled :
# Define model parameters
params = {
    'hidden_layer_sizes': (100, 50),    # Two hidden layers with 100 and 50 neurons
    'activation': 'relu',               # Activation function for the hidden layers:
    'solver': 'adam',                  # Optimization algorithm
    'random_state': seed               # Random seed for reproducibility
}

# Create and train model
nnet_train = MLPClassifier(**params).fit(X_train, y_train)

# Save to external file
joblib.dump(nnet_train, models_folder + '/' + 'nnet_train.pkl')
print(f'{nnet_train.__class__.__name__} model exported to {models_folder} folder')
else:
# Load model from external file
nnet_train = joblib.load(models_folder + '/' + 'nnet_train.pkl')
print(f'{nnet_train.__class__.__name__} model loaded from {models_folder} folder')

# Call previous function to show used parameters
show_parameters(nnet_train)
```

MLPClassifier model loaded from Models folder

Parameters:

```
activation: relu
alpha: 0.0001
batch_size: auto
beta_1: 0.9
beta_2: 0.999
early_stopping: False
epsilon: 1e-08
hidden_layer_sizes: (100, 50)
learning_rate: constant
learning_rate_init: 0.001
max_fun: 15000
max_iter: 200
momentum: 0.9
n_iter_no_change: 10
nesterovs_momentum: True
power_t: 0.5
random_state: 42
shuffle: True
solver: adam
tol: 0.0001
validation_fraction: 0.1
verbose: False
warm_start: False
```

Model Main Metrics

```
In [57]: # Call function to show main metrics
model_metrics(nnnet_train, X_test, y_test, styled=True)
```

Out[57]: **MLPClassifier Training**

metric	value
ROC AUC:	0.857700
Accuracy:	0.857400
Log loss:	5.140400
Kappa:	0.714900
MCC:	0.719900
F1:	0.864500
Sensitivity:	0.915300
Specificity:	0.800100

```
In [58]: # Call function to show confusion matrix
confusion_matrix_table(nnet_train, X_test, y_test)
```

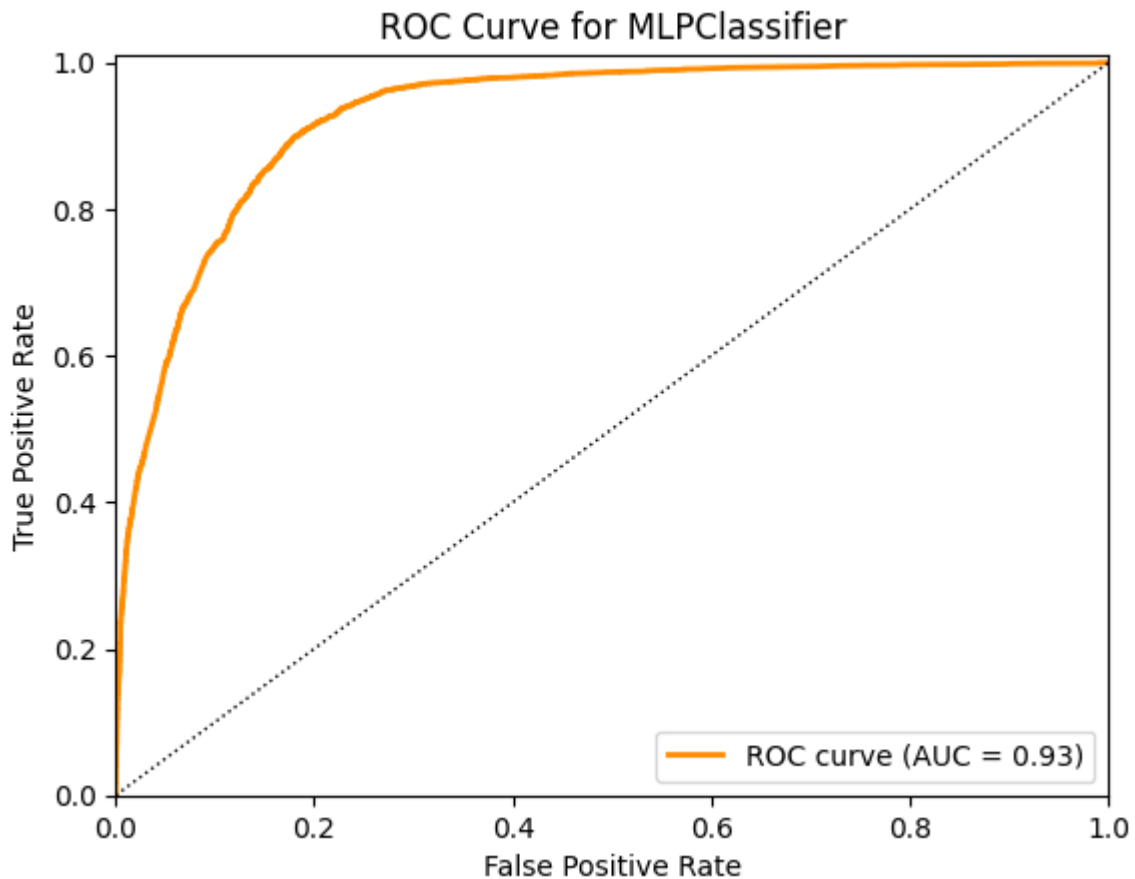
Out[58]: **Model: MLPClassifier**

Predicted: 1 0

Actual Label:

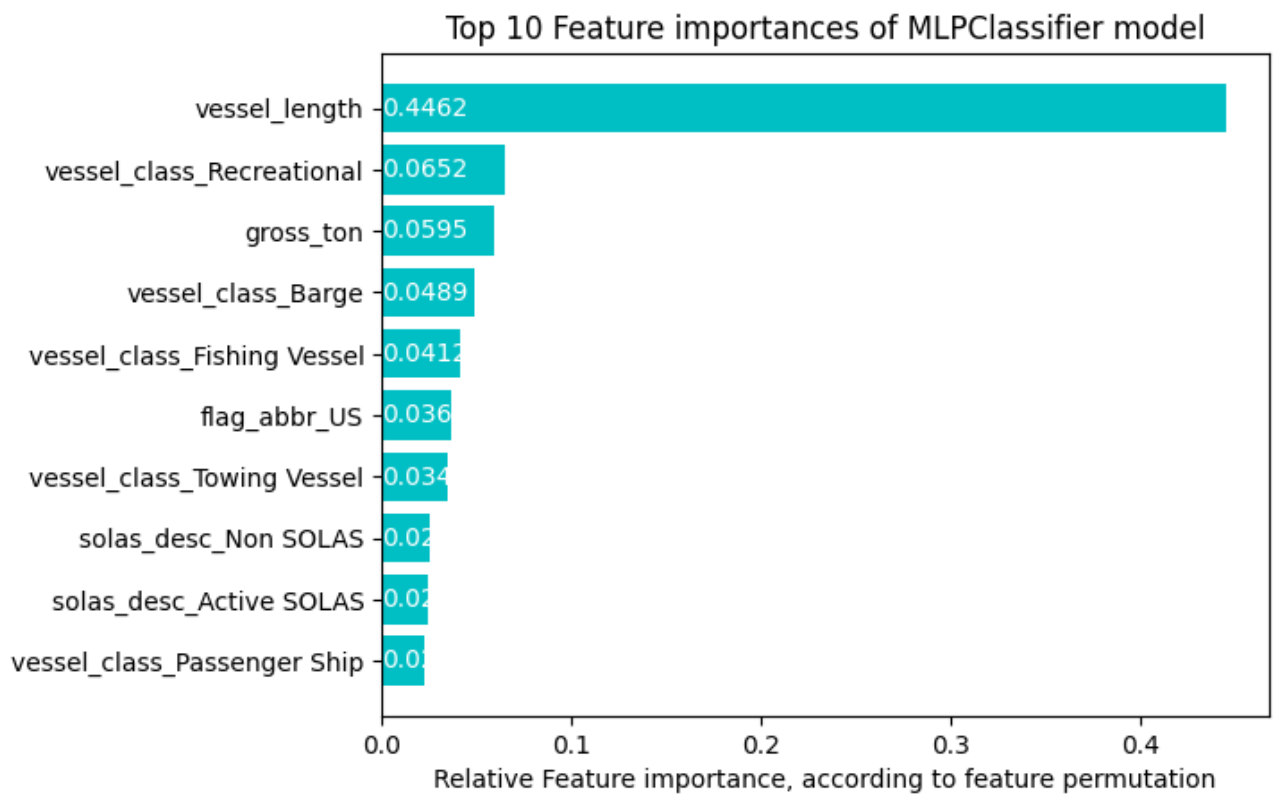
1	9996	925
0	2208	8839

```
In [59]: # Call function to plot Receiver Operating Characteristic (ROC) curve
plot_roc_curve(nnet_train, X_test, y_test)
```



Feature importance (permutation)

```
In [60]: # Call previous functions for plotting top 10 importances according to feature permutation
plot_importances(permutation_importances(nnet_train, X_train, y_train), top=10)
```



2.3.4. Classification and Regression Trees (CART)

Model Training

```
In [61]: if train_model_enabled :
# Define model parameters
params = {
    'criterion': 'gini',                # Criteria
    'random_state': seed                # Random seed for reproducibility
}

# Create and train model
cart_train = DecisionTreeClassifier(**params).fit(X_train, y_train)

# Save to external file
joblib.dump(cart_train, models_folder + '/' + 'cart_train.pkl')
print(f'{cart_train.__class__.__name__} model exported to {models_folder} folder')
else:
# Load model from external file
cart_train = joblib.load(models_folder + '/' + 'cart_train.pkl')
print(f'{cart_train.__class__.__name__} model loaded from {models_folder} folder')

# Call previous function to show used parameters
show_parameters(cart_train)
```

DecisionTreeClassifier model loaded from Models folder

Parameters:

```
ccp_alpha: 0.0
class_weight: None
criterion: gini
max_depth: None
max_features: None
max_leaf_nodes: None
min_impurity_decrease: 0.0
min_samples_leaf: 1
min_samples_split: 2
min_weight_fraction_leaf: 0.0
monotonic_cst: None
random_state: 42
splitter: best
```

Model Main Metrics

```
In [62]: # Call function to show main metrics
model_metrics(cart_train, X_test, y_test, styled=True)
```

Out[62]: **DecisionTreeClassifier Training**

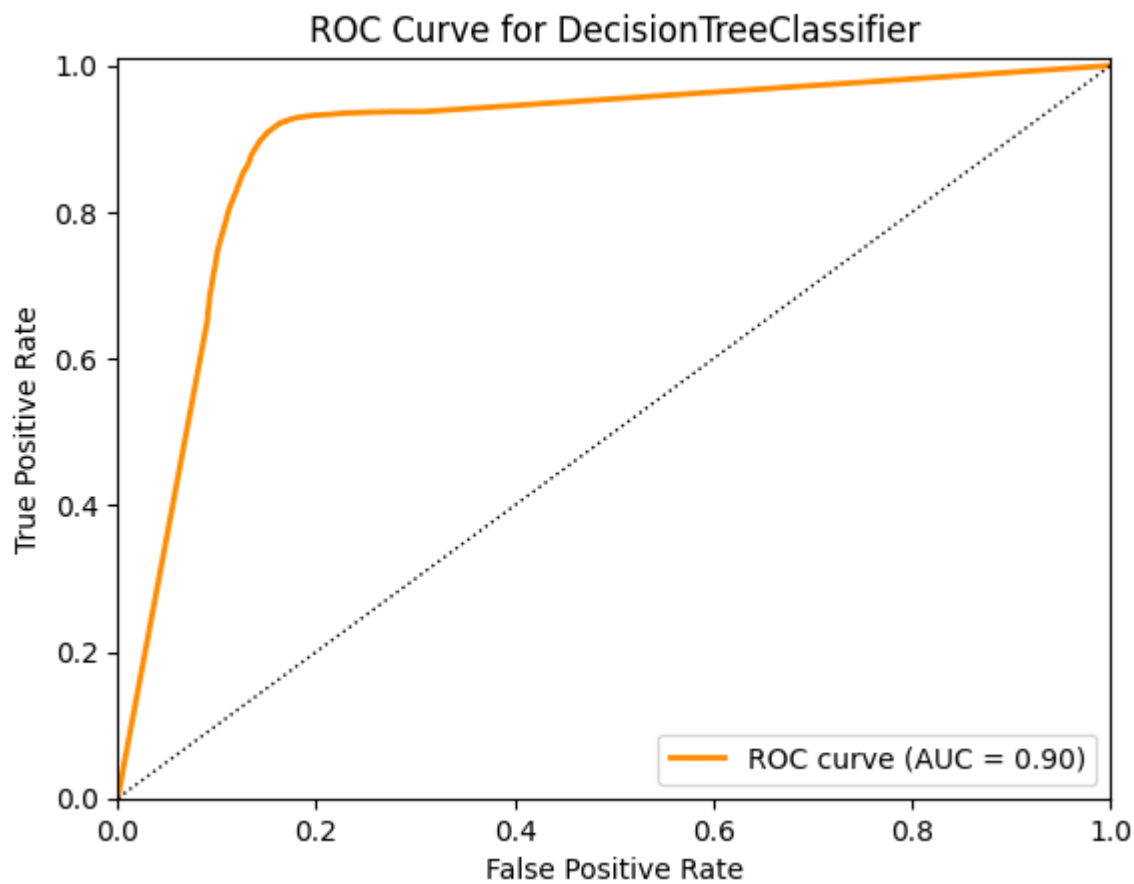
metric	value
ROC AUC:	0.879000
Accuracy:	0.878900
Log loss:	4.366000
Kappa:	0.757800
MCC:	0.759200
F1:	0.881800
Sensitivity:	0.908800
Specificity:	0.849300

```
In [63]: # Call function to show confusion matrix
confusion_matrix_table(cart_train, X_test, y_test)
```

Out[63]: **Model: DecisionTreeClassifier**

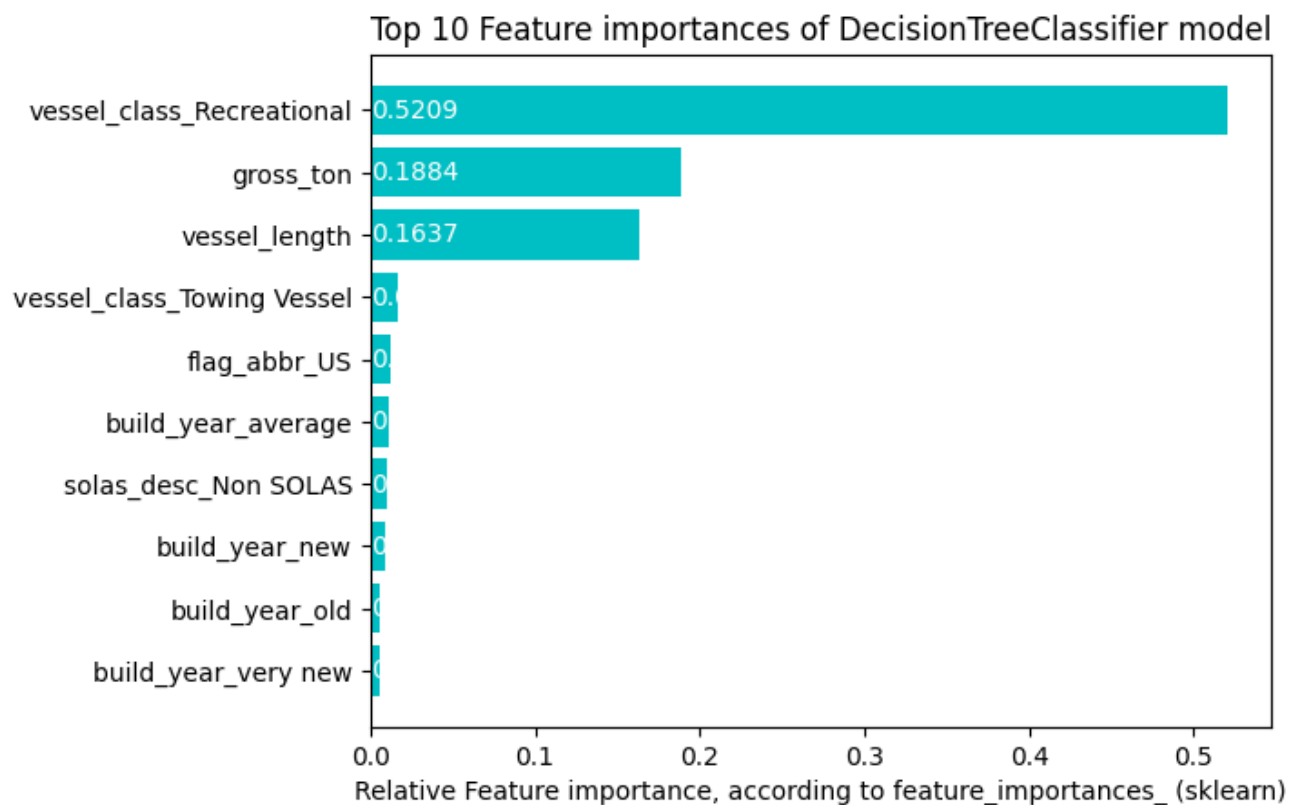
Predicted:	1	0
Actual Label:		
1	9925	996
0	1665	9382

```
In [64]: # Call function to plot Receiver Operating Characteristic (ROC) curve
plot_roc_curve(cart_train, X_test, y_test)
```



Feature importance

In [65]: `# Call previous functions for plotting top 10 importances according to feature_importances_`
`plot_importances(sklearn_importances(cart_train), top=10)`



2.3.5. Logistic Regression

Model Training

```
In [66]: if train_model_enabled :
# Define model parameters
params = {
    'random_state': seed                # Random seed for reproducibility
}

# Create and train model
rl_train = LogisticRegression(**params).fit(X_train, y_train)

# Save to external file
joblib.dump(rl_train, models_folder + '/' + 'rl_train.pkl')
print(f'{rl_train.__class__.__name__} model exported to {models_folder} folder')
else:
# Load model from external file
rl_train = joblib.load(models_folder + '/' + 'rl_train.pkl')
print(f'{rl_train.__class__.__name__} model loaded from {models_folder} folder')

# Call previous function to show used parameters
show_parameters(rl_train)
```

LogisticRegression model loaded from Models folder

Parameters:

```
C: 1.0
class_weight: None
dual: False
fit_intercept: True
intercept_scaling: 1
l1_ratio: None
max_iter: 100
multi_class: auto
n_jobs: None
penalty: l2
random_state: 42
solver: lbfgs
tol: 0.0001
verbose: 0
warm_start: False
```

Model Main Metrics

```
In [67]: # Call function to show main metrics
model_metrics(rl_train, X_test, y_test, styled=True)
```

Out[67]: **LogisticRegression Training**

metric	value
ROC AUC:	0.842600
Accuracy:	0.842100
Log loss:	5.691700
Kappa:	0.684500
MCC:	0.695400
F1:	0.854200
Sensitivity:	0.930100
Specificity:	0.755000

```
In [68]: # Call function to show confusion matrix
confusion_matrix_table(rl_train, X_test, y_test)
```

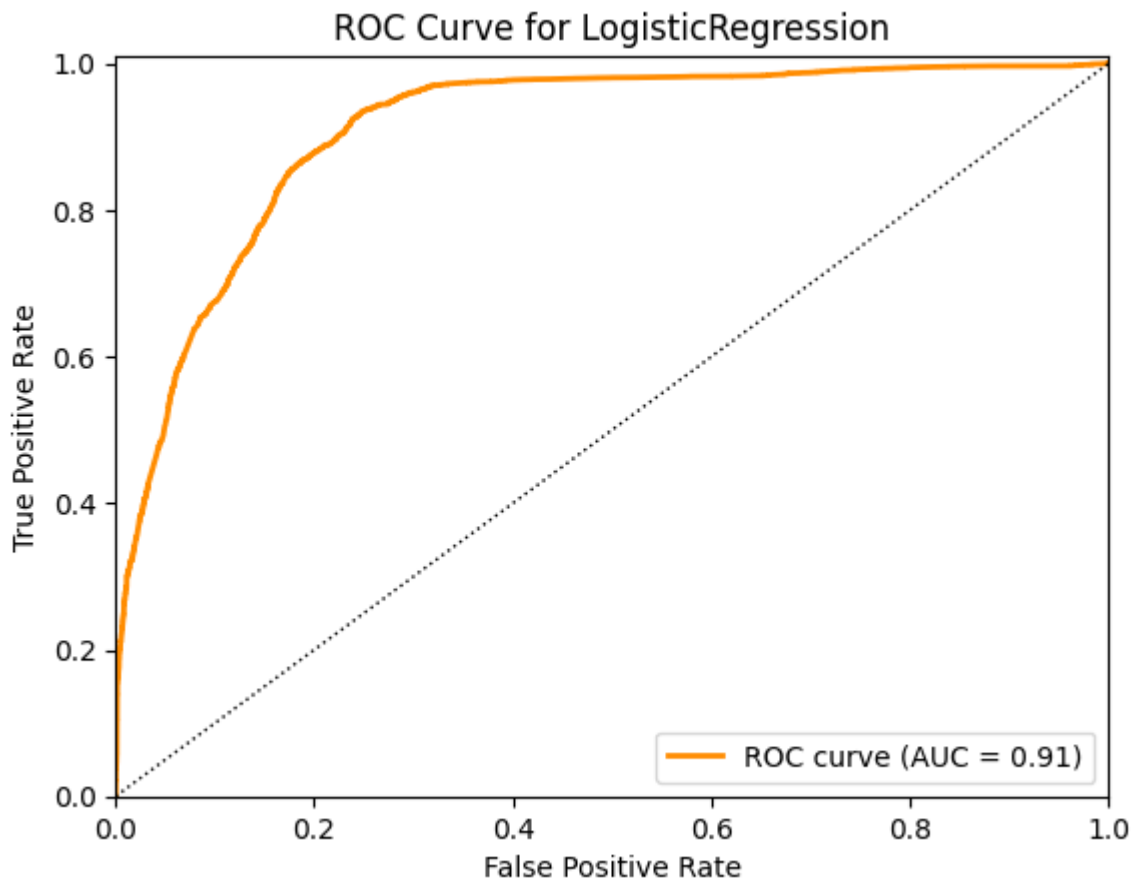
Out[68]: **Model: LogisticRegression**

Predicted: 1 0

Actual Label:

1	10158	763
0	2706	8341

```
In [69]: # Call function to plot Receiver Operating Characteristic (ROC) curve
plot_roc_curve(rl_train, X_test, y_test)
```



3. Model Comparison

```
In [70]: # Midpoint milestone: Load all trained models from here

# All model list
models = ['nb_train', 'bnb_train', 'GBM_train',
          'XGB_train', 'rf_train', 'svm_train',
          'nnet_train', 'cart_train', 'rl_train']

# Load all models and check
for model in models:
    globals()[model] = joblib.load(models_folder + '/' + model + '.pkl')
    print(f'{model} loaded from {models_folder} folder')
```

```
nb_train loaded from Models folder  
bnb_train loaded from Models folder  
GBM_train loaded from Models folder  
XGB_train loaded from Models folder  
rf_train loaded from Models folder  
svm_train loaded from Models folder  
nnet_train loaded from Models folder  
cart_train loaded from Models folder  
rl_train loaded from Models folder
```

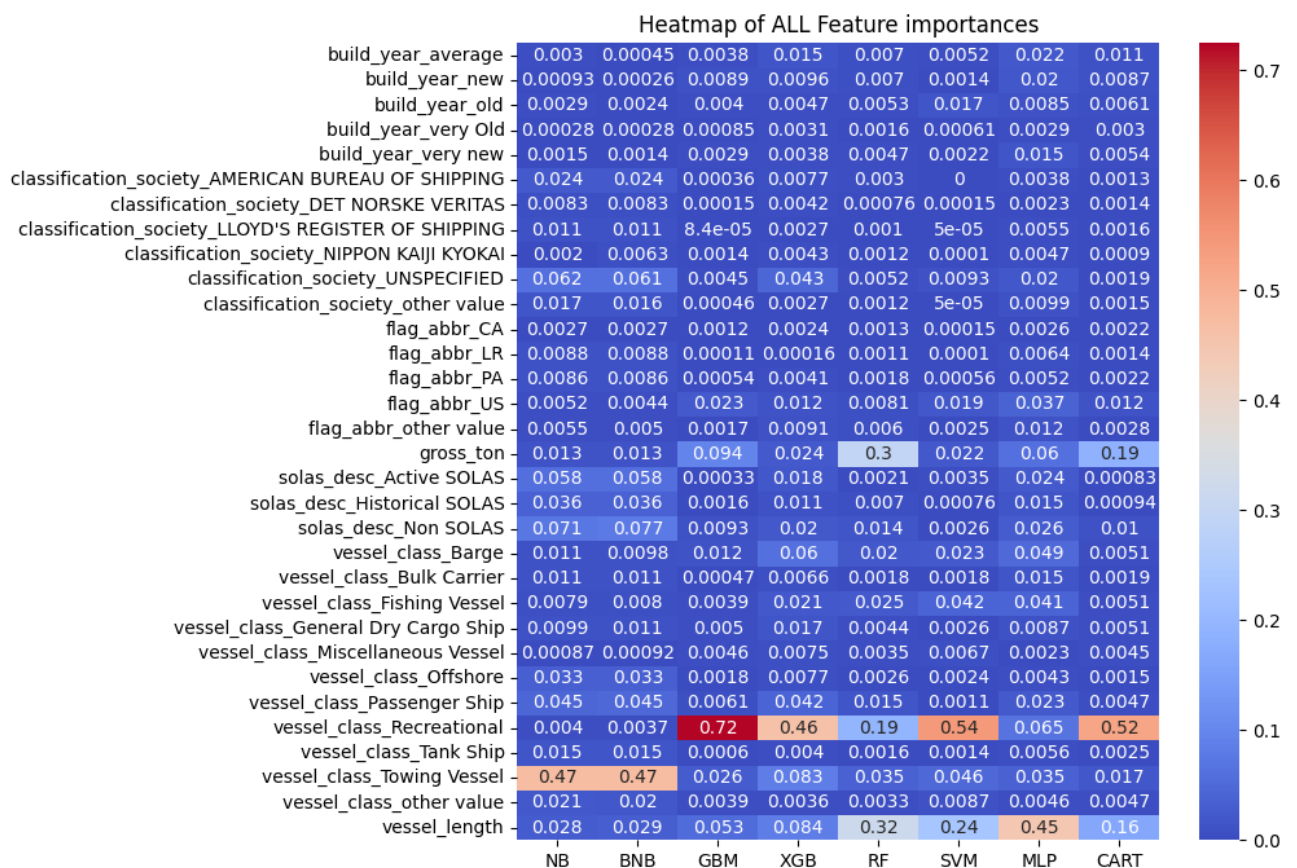
3.1. Feature importance comparison

Heatmap of ALL Feature importances for trained models

```
In [71]: # Concatenate variable importance values by variable_name as index
all_models_importances = pd.concat([
    permutation_importances(nb_train, X_train, y_train).set_index('variable_name'),
    permutation_importances(bnb_train, X_train, y_train).set_index('variable_name'),
    sklearn_importances(GBM_train).set_index('variable_name'),
    sklearn_importances(XGB_train).set_index('variable_name'),
    sklearn_importances(rf_train).set_index('variable_name'),
    svm_importances.set_index('variable_name'),
    permutation_importances(nnet_train, X_train, y_train).set_index('variable_name'),
    sklearn_importances(cart_train).set_index('variable_name')
], axis=1).sort_index()

# Model names for column naming
all_models_importances.columns = ('NB', 'BNB', 'GBM', 'XGB', 'RF', 'SVM', 'MLP', 'CART')

# Heatmap creation
plt.figure(figsize=(8, 8))
sns.heatmap(all_models_importances, annot=True, cmap='coolwarm')
plt.title('Heatmap of ALL Feature importances')
plt.ylabel('')
plt.show()
```



Heatmap of GROUPED Feature importances for trained models

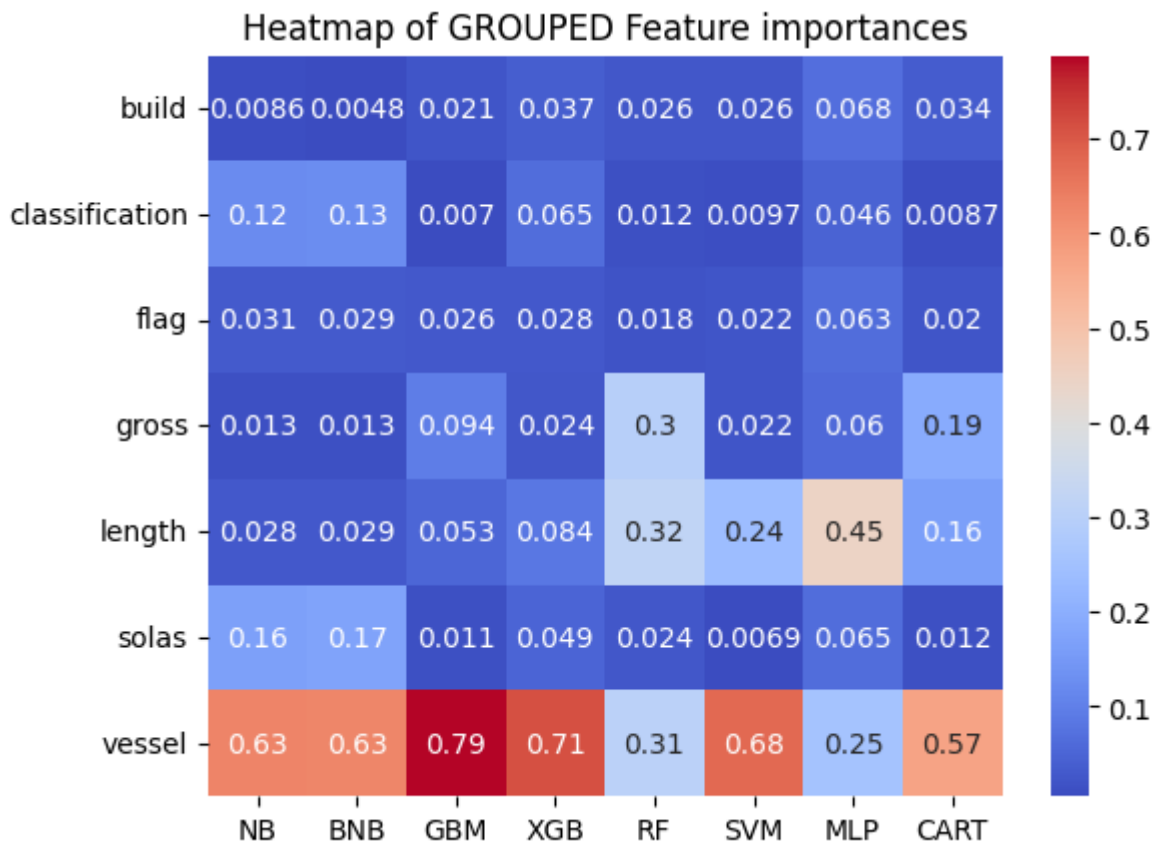
Same as previous but grouping by variable

```
In [72]: # Rename vessel_length for better distinction
all_models_importances.index = all_models_importances.index.str.replace('vessel_length',

# Create a new variable for the group
all_models_importances['group'] = all_models_importances.index.to_series().str.split('_')

# Sum calculated vertically by group
all_models_importances_grouped = all_models_importances.groupby('group').sum()
```

```
# Heatmap creation
plt.figure()
sns.heatmap(all_models_importances_grouped , annot=True, cmap='coolwarm')
plt.title('Heatmap of GROUPED Feature importances')
plt.ylabel('')
plt.show()
```



3.2. Performance metrics of the models

Metrics comparison tables

```
In [73]: def comparision_table (X, y, title = 'Model performance comparison'):
# Build table, adding model_metrics function for each model. Set index and transpose
df = pd.DataFrame({
    model.split('_')[0].upper(): model_metrics(eval(model), X, y)['value'] for model
}).set_index(model_metrics(eval(models[0]), X, y)['metric']).T

# Sort table by ROC AUC
df = df.sort_values(['ROC AUC:'], ascending=[False])

# Add title
df.columns = pd.MultiIndex.from_tuples([(title, col) for col in df.columns])

return df
```

```
In [74]: # Call function for training sample
comparision_table (X_train, y_train, 'Model performance comparison for training sample')
```

Out[74]:

Model performance comparison for training sample								
	ROC AUC:	Accuracy:	Log loss:	Kappa:	MCC:	F1:	Sensitivity:	Specificity:
RF	0.9515	0.9515	1.7487	0.903	0.9037	0.9525	0.9717	0.9312
CART	0.9515	0.9515	1.7479	0.903	0.9034	0.9523	0.9663	0.9367
GBM	0.8662	0.8662	4.8211	0.7324	0.7366	0.8732	0.9195	0.8128
NNET	0.8638	0.8639	4.9068	0.7277	0.7328	0.8716	0.9232	0.8044
XGB	0.8491	0.8492	5.4344	0.6984	0.705	0.8591	0.9179	0.7804
SVM	0.8479	0.848	5.477	0.696	0.7087	0.8613	0.9423	0.7536
RL	0.838	0.8381	5.8359	0.6761	0.6874	0.8517	0.9285	0.7474
BNB	0.6734	0.6731	11.7835	0.3466	0.3855	0.5821	0.4547	0.892
NB	0.6733	0.673	11.788	0.3463	0.3851	0.5821	0.455	0.8915

```
In [75]: # Call function for test sample
          comparision_table (X_test, y_test, 'Model performance comparison for test sample')
```

Out[75]:

Model performance comparison for test sample								
	ROC AUC:	Accuracy:	Log loss:	Kappa:	MCC:	F1:	Sensitivity:	Specificity:
RF	0.883	0.8828	4.2249	0.7657	0.768	0.8866	0.9213	0.8447
CART	0.879	0.8789	4.366	0.7578	0.7592	0.8818	0.9088	0.8493
GBM	0.8651	0.8648	4.8713	0.7299	0.734	0.871	0.9176	0.8127
NNET	0.8577	0.8574	5.1404	0.7149	0.7199	0.8645	0.9153	0.8001
XGB	0.8515	0.8511	5.3668	0.7024	0.7088	0.8597	0.9175	0.7855
SVM	0.8492	0.8487	5.4538	0.6977	0.7101	0.8609	0.9416	0.7569
RL	0.8426	0.8421	5.6917	0.6845	0.6954	0.8542	0.9301	0.755
BNB	0.6738	0.6751	11.7099	0.3486	0.3888	0.5799	0.451	0.8967
NB	0.6737	0.6749	11.7165	0.3482	0.3882	0.58	0.4514	0.8959

Graphic charts for metrics

```
In [76]: # Create dataframe for test sample metrics
          df = comparision_table(X_test, y_test)

          # Drop title
          df.columns = df.columns.get_level_values(-1)

          # Drop 'separator' column
          df = df.drop(df.columns[-3], axis=1)

          # Create subplots with a 4x2 grid
          fig, axes = plt.subplots(nrows=4, ncols=2, figsize=(8, 8))

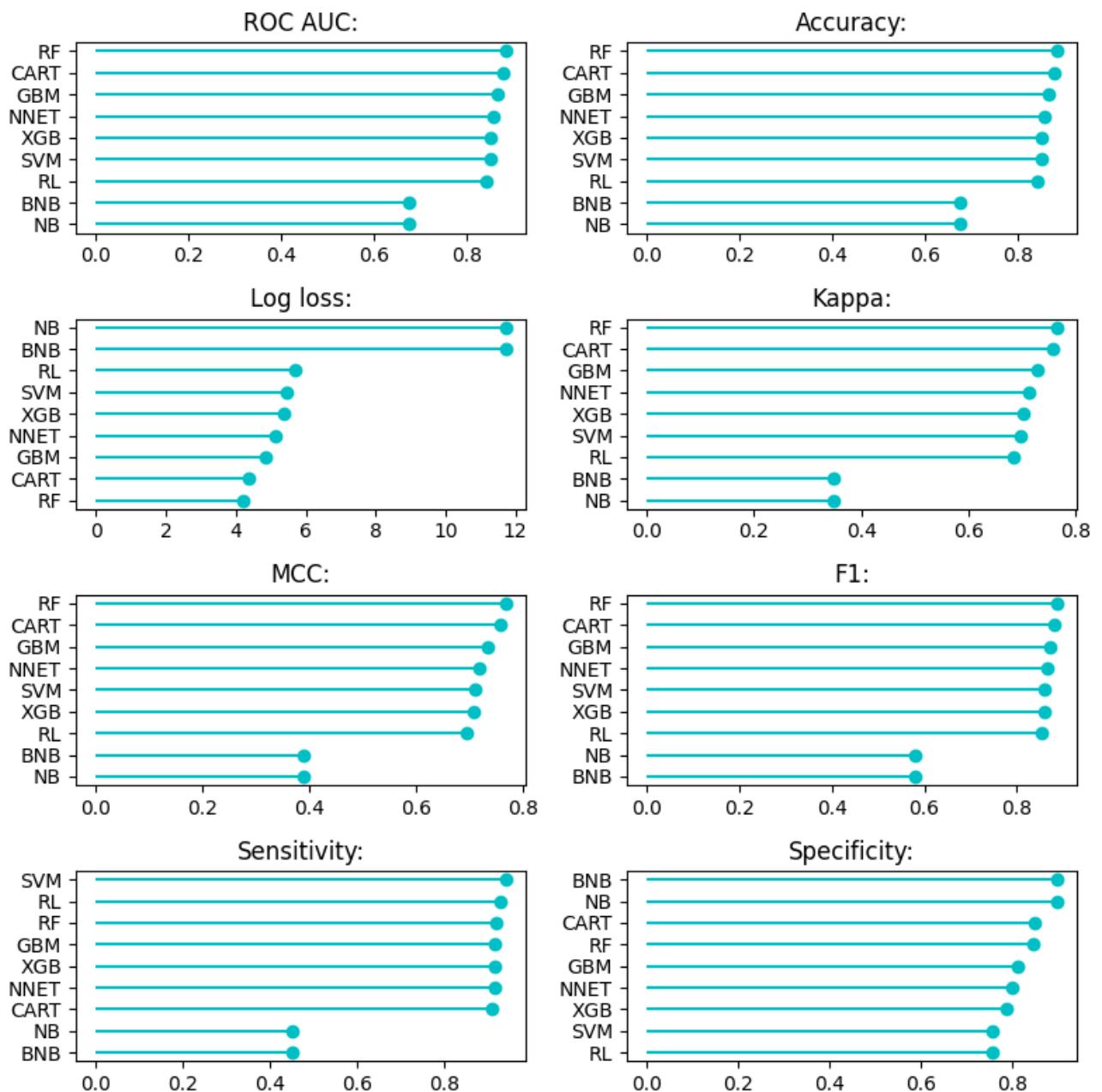
          # Flatten the axes array for easy iteration
          axes = axes.flatten()
```

```
# Function to create a horizontal lollipop chart on a specific subplot
def plot_lollipop(ax, df, metric):
    # Sort models by the metric
    df_sorted = df.sort_values(by=metric)

    # Create the Lollipop chart
    ax.hlines(y=df_sorted.index, xmin=0, xmax=df_sorted[metric], color='#00bfc4')
    ax.plot(df_sorted[metric], df_sorted.index, 'o', color='#00bfc4')
    ax.set_title(f'{metric}')

# Create lollipop charts for each metric
for ax, metric in zip(axes, df.columns):
    plot_lollipop(ax, df, metric)

# Adjust the space between subplots
plt.tight_layout()
plt.show()
```



ROC Curves Comparison

```
In [77]: # Adjust the figure size
plt.figure(figsize=(8, 6))

for model in models:
```

```

# Predicted probabilities for the positive class
y_score = eval(model).predict_proba(X_test)[:, 1]

# Calculate ROC curve
fpr, tpr, _ = roc_curve(y_test, y_score)

# Calculate AUC (Area Under Curve)
roc_auc = auc(fpr, tpr)

# Plot ROC Curve
plt.plot(fpr, tpr, lw=2, label=f'{model.split("_")[0].upper()} (AUC = %0.2f)' % roc_auc)

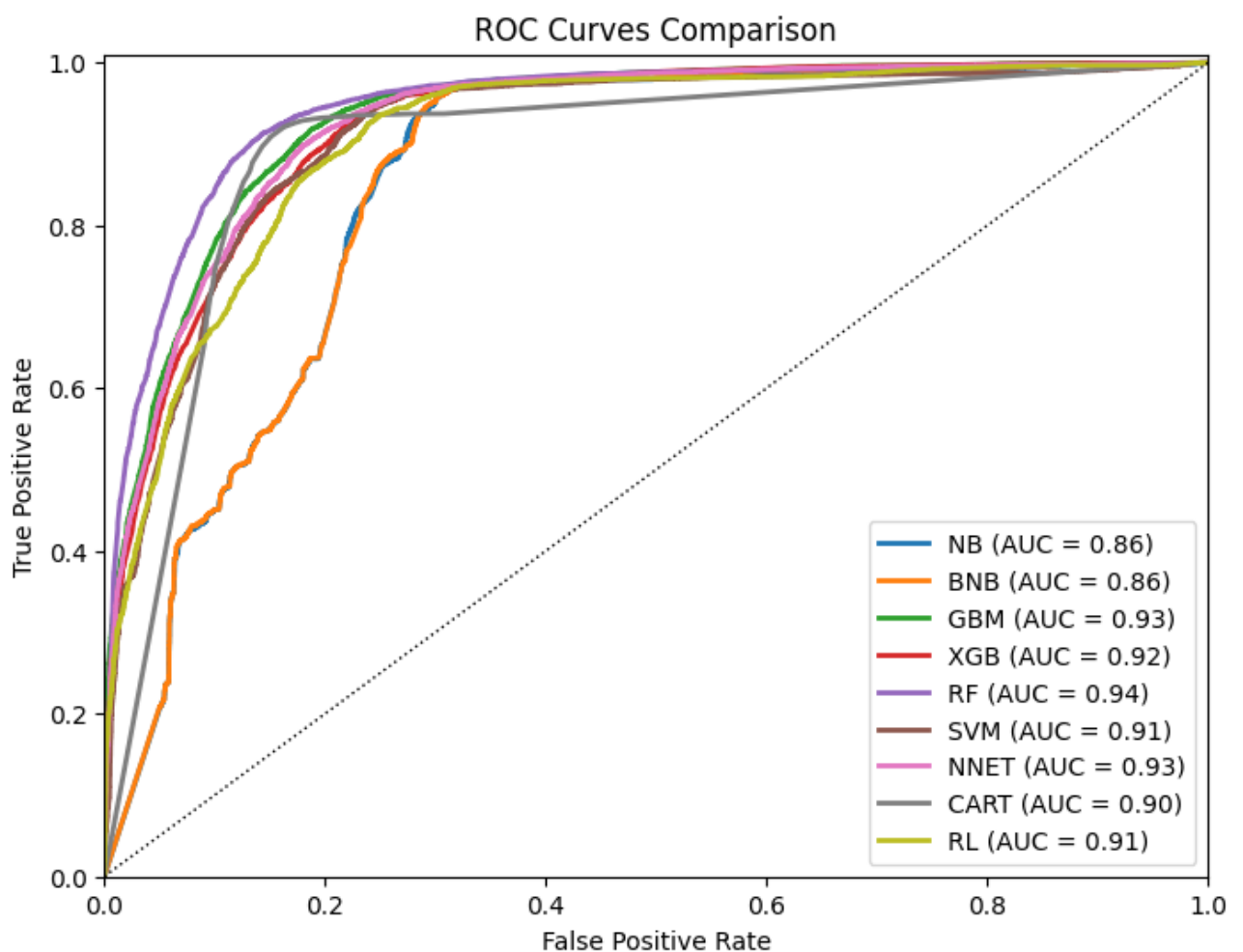
# Plot diagonal line (random classifier)
plt.plot([0, 1], [0, 1], color='black', lw=1, linestyle='dotted')

# Set plot limits, Labels, and title
plt.xlim([0, 1])
plt.ylim([0, 1.01])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves Comparison')

# Display Legend
plt.legend(loc="lower right")

# Show plot
plt.show()

```



3.3. Accuracy for Cross Validation (10 splits)

```
In [78]: if file_export_enabled :
```

```

# Calculate scores for accuracy for all models
kfold = KFold(n_splits=10, shuffle=True, random_state=seed)
accuracy_scores = []
for model in models:
    accuracy_scores.append(cross_val_score(eval(model), X_train, y_train, scoring='a

# Store scores in a pandas dataframe and export to external file
accuracy_scores = pd.DataFrame(accuracy_scores)
accuracy_scores.columns = accuracy_scores.columns.astype(str)
accuracy_scores.reset_index().to_feather(datasets_folder + '/' + 'accuracy_scores.fe

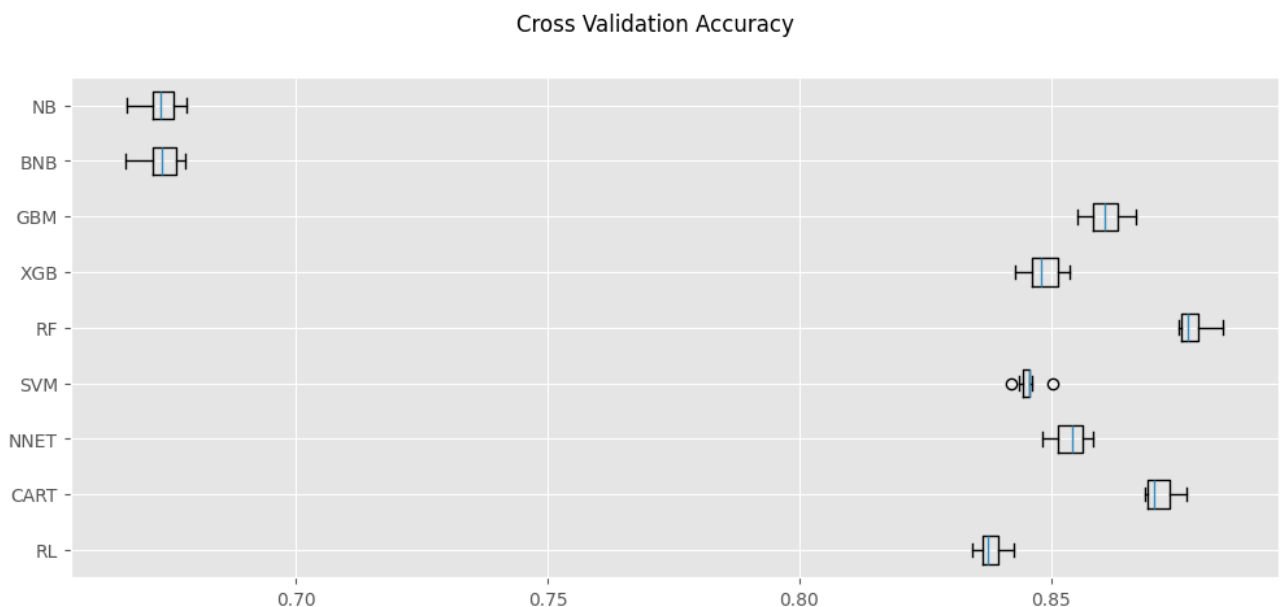
else:
    # Load this dataframe otherwise
    accuracy_scores = pd.read_feather(datasets_folder + '/' + 'accuracy_scores.feather')

```

```

In [79]: # Boxplots of accuracy values in cross validation
plt.style.use('ggplot')
fig = plt.figure(figsize=(12,5))
fig.suptitle('Cross Validation Accuracy')
ax = fig.add_subplot(111)
plt.boxplot(accuracy_scores[:, :-1].T.iloc[1:].reset_index(drop=True), vert=False)
ax.set_yticklabels([model.split('_')[0].upper() for model in models][::-1])
plt.show()

```



Statistical hypothesis test

```

In [80]: # Convert score data to list
score_list = accuracy_scores.values.tolist()

# Initiate table
table=[]
for i in range(len(score_list)):
    table.append([])
for i in range(len(score_list)):
    for j in range(len(score_list)):
        table[i].append(0)

# Populate values
for i in range(len(score_list)):
    for j in range(i+1, len(score_list)):
        stat, p = ttest_rel(score_list[i], score_list[j])

alpha = 0.05

```

```

table[i][j]=np.round(p,3) # upper diagonal
table[j][i]=np.round(stat,3) # lower diagonal
table[i][i]=''

print('The following table shows the comparison of the statistics and the p-values of the models')
print('The value of the statistic on the lower diagonal and the p-value on the upper diagonal')

# Show table, including model names
table = pd.DataFrame(table).set_index(pd.Index([model.split('_')[0].upper() for model in models]))
table.columns = [model.split('_')[0].upper() for model in models]
table

```

The following table shows the comparison of the statistics and the p-values of the models. The value of the statistic on the lower diagonal and the p-value on the upper diagonal.

Out[80]:

	NB	BNB	GBM	XGB	RF	SVM	NNET	CART	RL
NB		0.34	0.058	0.123	0.142	0.194	0.21	0.216	0.246
BNB	-1.001		0.005	0.067	0.101	0.166	0.188	0.199	0.235
GBM	-2.138	-3.536		0.407	0.298	0.367	0.35	0.33	0.361
XGB	-1.682	-2.055	-0.865		0.21	0.348	0.332	0.312	0.352
RF	-1.594	-1.807	-1.097	-1.339		0.53	0.407	0.352	0.394
SVM	-1.393	-1.495	-0.944	-0.984	-0.651		0.301	0.279	0.354
NNET	-1.341	-1.412	-0.98	-1.019	-0.866	-1.09		0.258	0.382
CART	-1.321	-1.376	-1.024	-1.064	-0.975	-1.145	-1.2		0.535
RL	-1.231	-1.265	-0.958	-0.977	-0.89	-0.972	-0.914	-0.642	0.000

3.4. Learning curves

```

In [81]: # Select models for this comparison
selected_models = ['GBM_train', 'XGB_train', 'rf_train', 'cart_train']

# Function to plot learning curves
def plot_learning_curve(model, X, y, cv, train_sizes=np.linspace(.1, 1.0, 5)):
    plt.figure()
    plt.title(f'Learning Curve of {model}[:70]')
    plt.xlabel("Training examples")
    plt.ylabel("Score")

    train_sizes, train_scores, test_scores = learning_curve(
        model, X, y, cv=cv, train_sizes=train_sizes, n_jobs=n_jobs)
    train_scores_mean = np.mean(train_scores, axis=1)
    train_scores_std = np.std(train_scores, axis=1)
    test_scores_mean = np.mean(test_scores, axis=1)
    test_scores_std = np.std(test_scores, axis=1)

    plt.grid()

    plt.fill_between(train_sizes, train_scores_mean - train_scores_std,
                     train_scores_mean + train_scores_std, alpha=0.1, color="r")
    plt.fill_between(train_sizes, test_scores_mean - test_scores_std,
                     test_scores_mean + test_scores_std, alpha=0.1, color="g")
    plt.plot(train_sizes, train_scores_mean, 'o-', color="r",
             label="Training score")
    plt.plot(train_sizes, test_scores_mean, 'o-', color="g",
             label="Cross-validation score")

```

```

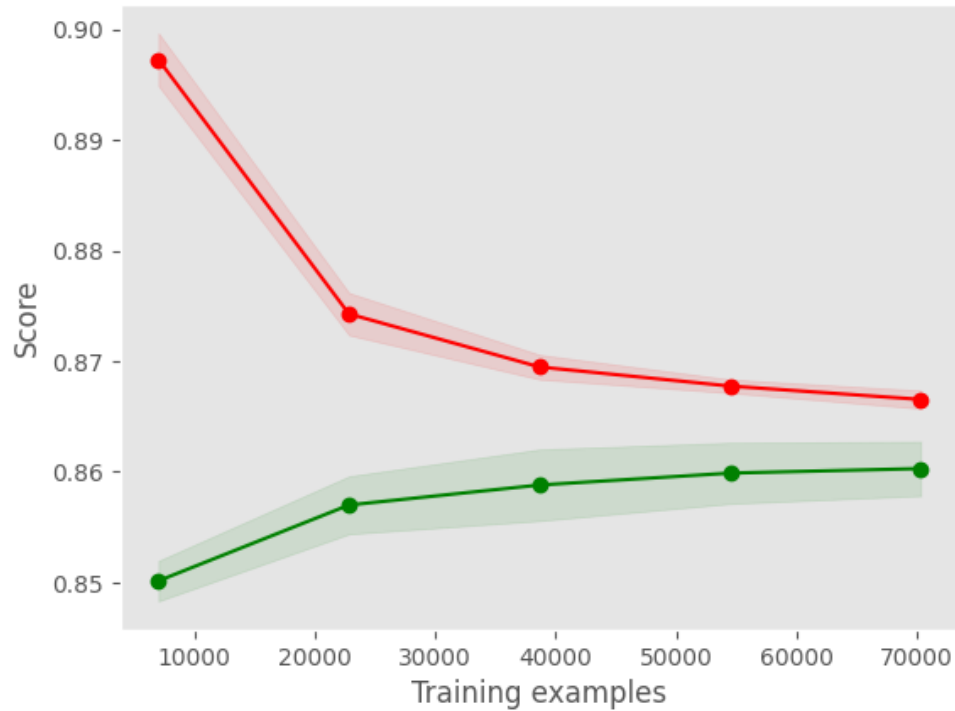
return plt

# Set sample split
cv = ShuffleSplit(n_splits=10, test_size=0.2, random_state=seed)

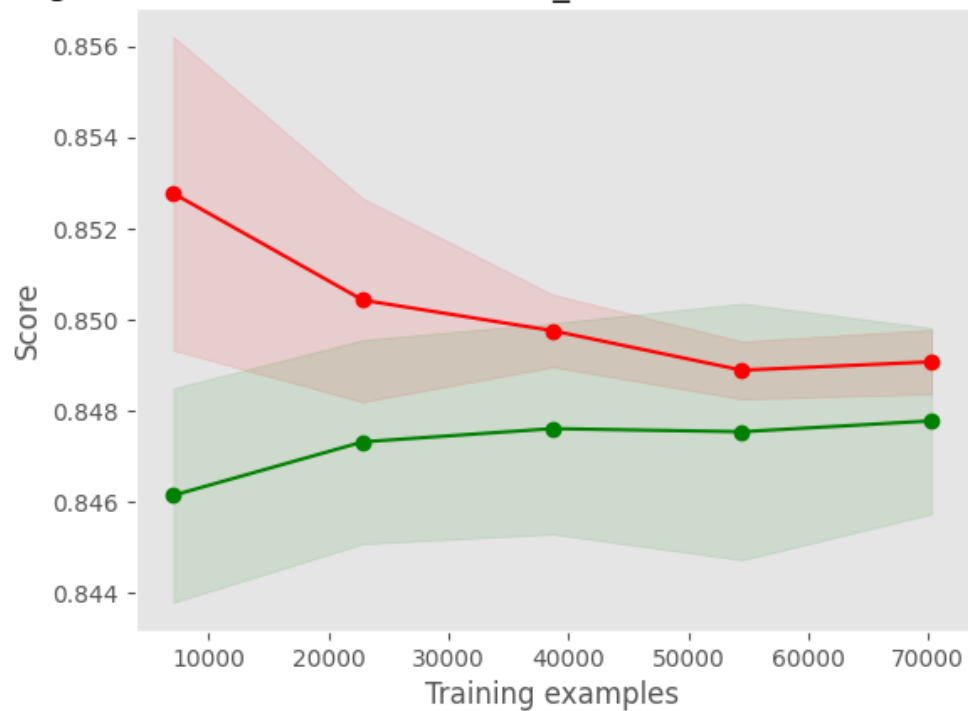
for model in selected_models:
    # Plot learning curves
    plot_learning_curve(eval(model), X_train, y_train, cv=cv)

```

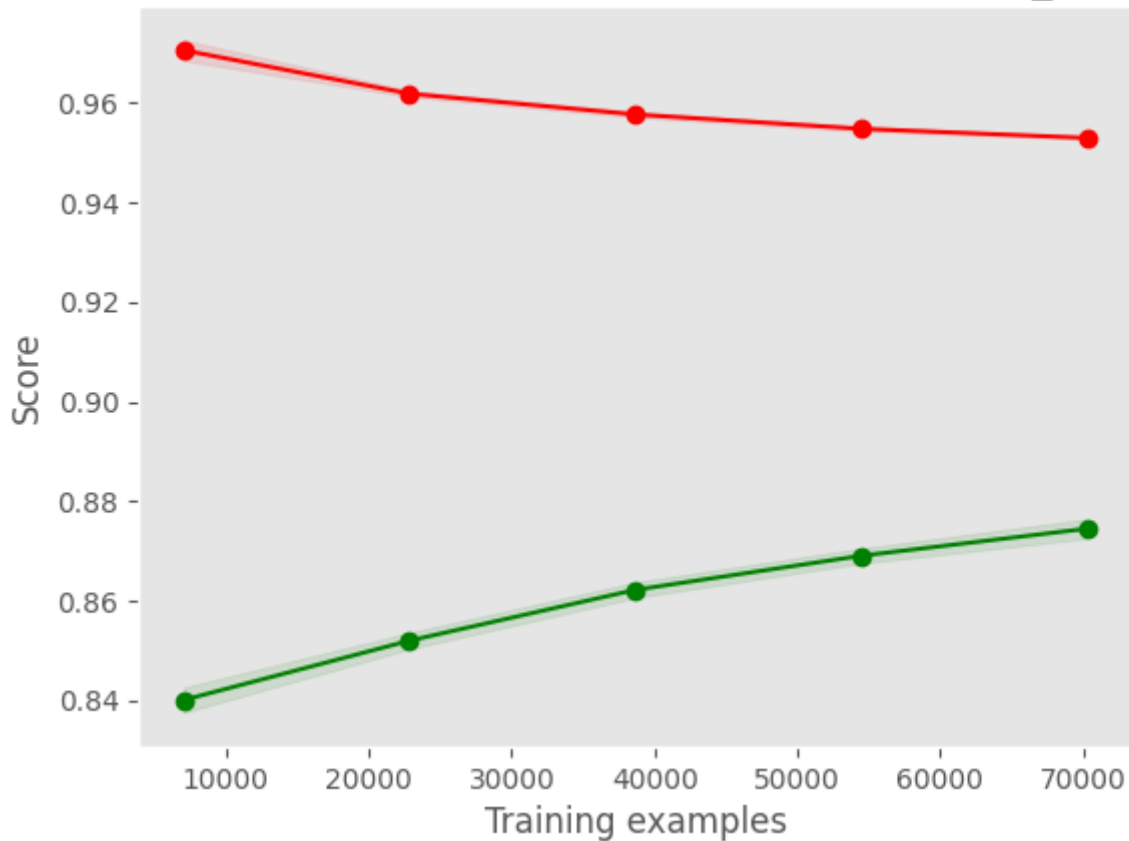
Learning Curve of GradientBoostingClassifier(n_estimators=500, random_



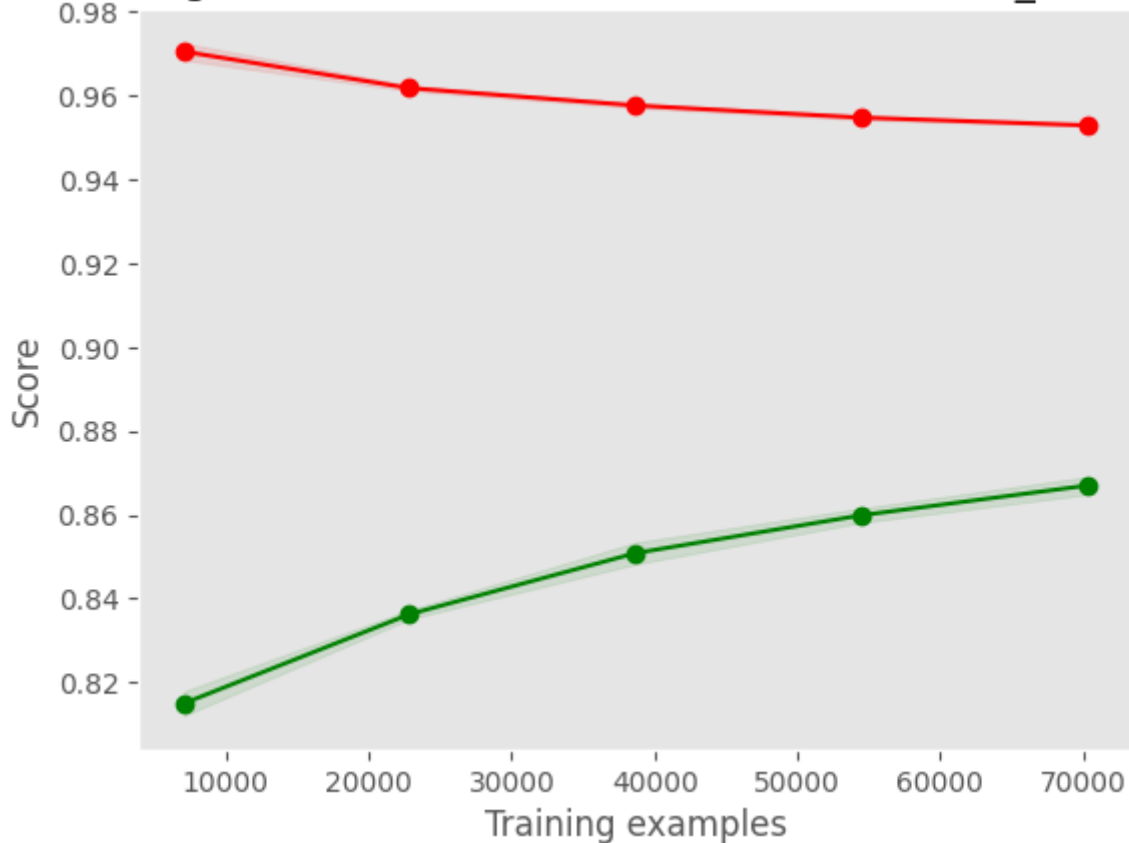
Learning Curve of XGBClassifier(base_score=None, booster=None, callback



Learning Curve of RandomForestClassifier(random_state=42)



Learning Curve of DecisionTreeClassifier(random_state=42)



4. Interpretability

```
In [82]: # Create dalex explainers dict for selected models
explainers = {}
for model in selected_models:
    explainers[model] = dx.Explainer(eval(model), X_train, y_train, label=model.split('_'
```

Preparation of a new explainer is initiated

```
-> data                : 87868 rows 32 cols
-> target variable     : Parameter 'y' was a pandas.Series. Converted to a numpy.ndarray.
-> target variable     : 87868 values
-> model_class         : sklearn.ensemble._gb.GradientBoostingClassifier (default)
-> label               : GBM
-> predict function    : <function yhat_proba_default at 0x000001E1C8F37880> will be used
(default)
-> predict function    : Accepts pandas.DataFrame and numpy.ndarray.
-> predicted values    : min = 0.00103, mean = 0.501, max = 0.997
-> model type          : classification will be used (default)
-> residual function   : difference between y and yhat (default)
-> residuals           : min = -0.995, mean = 2.86e-06, max = 0.99
-> model_info          : package sklearn
```

A new explainer has been created!

Preparation of a new explainer is initiated

```
-> data                : 87868 rows 32 cols
-> target variable     : Parameter 'y' was a pandas.Series. Converted to a numpy.ndarray.
-> target variable     : 87868 values
-> model_class         : xgboost.sklearn.XGBClassifier (default)
-> label               : XGB
-> predict function    : <function yhat_proba_default at 0x000001E1C8F37880> will be used
(default)
-> predict function    : Accepts pandas.DataFrame and numpy.ndarray.
-> predicted values    : min = 0.0123, mean = 0.501, max = 0.988
-> model type          : classification will be used (default)
-> residual function   : difference between y and yhat (default)
-> residuals           : min = -0.986, mean = 2.31e-05, max = 0.986
-> model_info          : package xgboost
```

A new explainer has been created!

Preparation of a new explainer is initiated

```
-> data                : 87868 rows 32 cols
-> target variable     : Parameter 'y' was a pandas.Series. Converted to a numpy.ndarray.
-> target variable     : 87868 values
-> model_class         : sklearn.ensemble._forest.RandomForestClassifier (default)
-> label               : RF
-> predict function    : <function yhat_proba_default at 0x000001E1C8F37880> will be used
(default)
-> predict function    : Accepts pandas.DataFrame and numpy.ndarray.
-> predicted values    : min = 0.0, mean = 0.507, max = 1.0
-> model type          : classification will be used (default)
-> residual function   : difference between y and yhat (default)
-> residuals           : min = -0.987, mean = -0.00612, max = 0.993
-> model_info          : package sklearn
```

A new explainer has been created!

Preparation of a new explainer is initiated

```
-> data                : 87868 rows 32 cols
-> target variable     : Parameter 'y' was a pandas.Series. Converted to a numpy.ndarray.
-> target variable     : 87868 values
-> model_class         : sklearn.tree._classes.DecisionTreeClassifier (default)
-> label               : CART
-> predict function    : <function yhat_proba_default at 0x000001E1C8F37880> will be used
(default)
-> predict function    : Accepts pandas.DataFrame and numpy.ndarray.
-> predicted values    : min = 0.0, mean = 0.501, max = 1.0
-> model type          : classification will be used (default)
```

```
-> residual function : difference between y and yhat (default)
-> residuals          : min = -0.985, mean = 3.84e-19, max = 0.992
-> model_info         : package sklearn
```

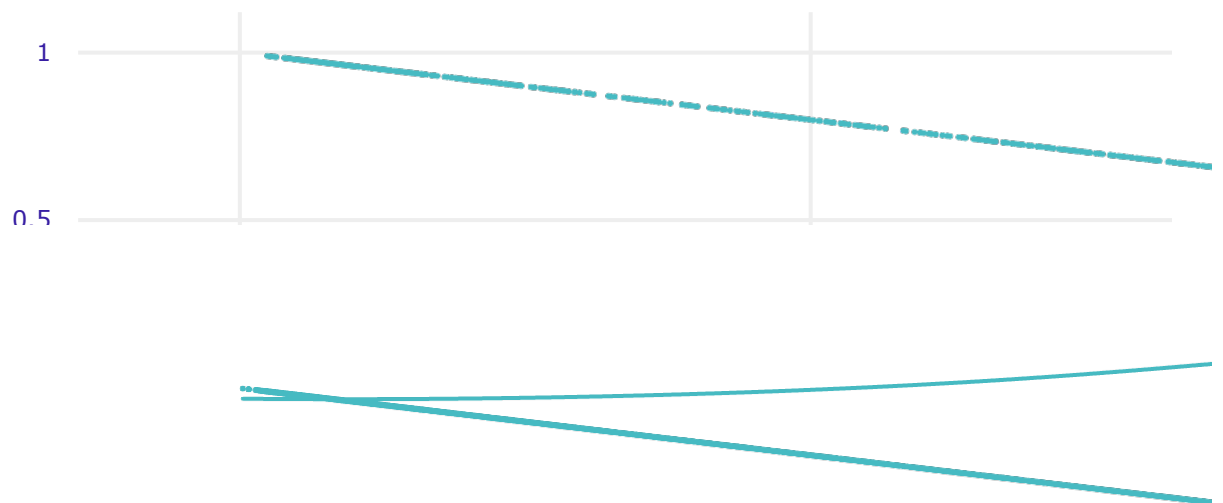
A new explainer has been created!

4.1. Residual Diagnostics

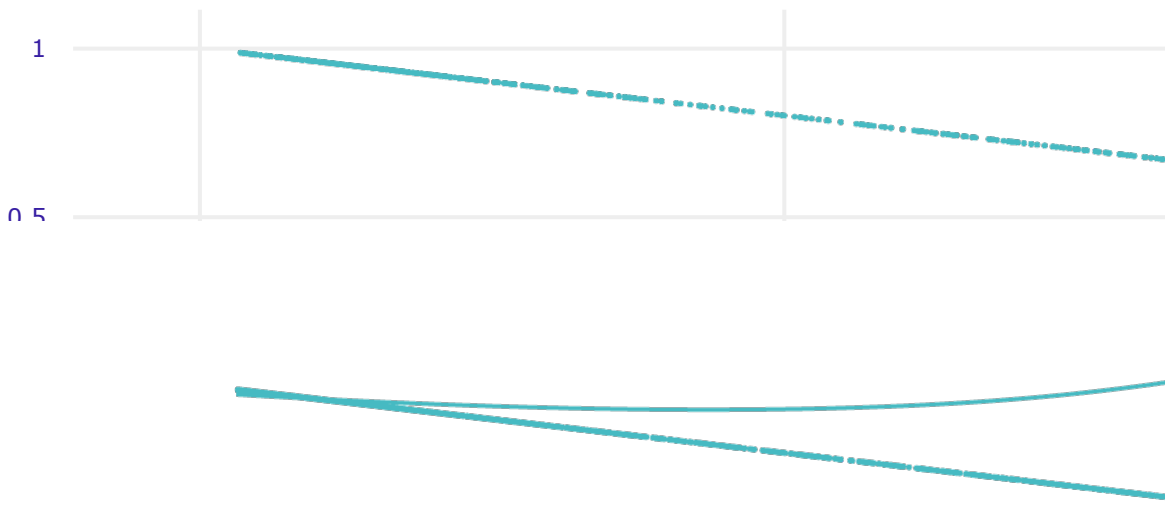
```
In [83]: # Ignore warnings
import warnings
warnings.filterwarnings("ignore")
```

```
In [84]: # Plot Residual Diagnostics
for explainer in explainers.values():
    explainer.model_diagnostics().plot()
```

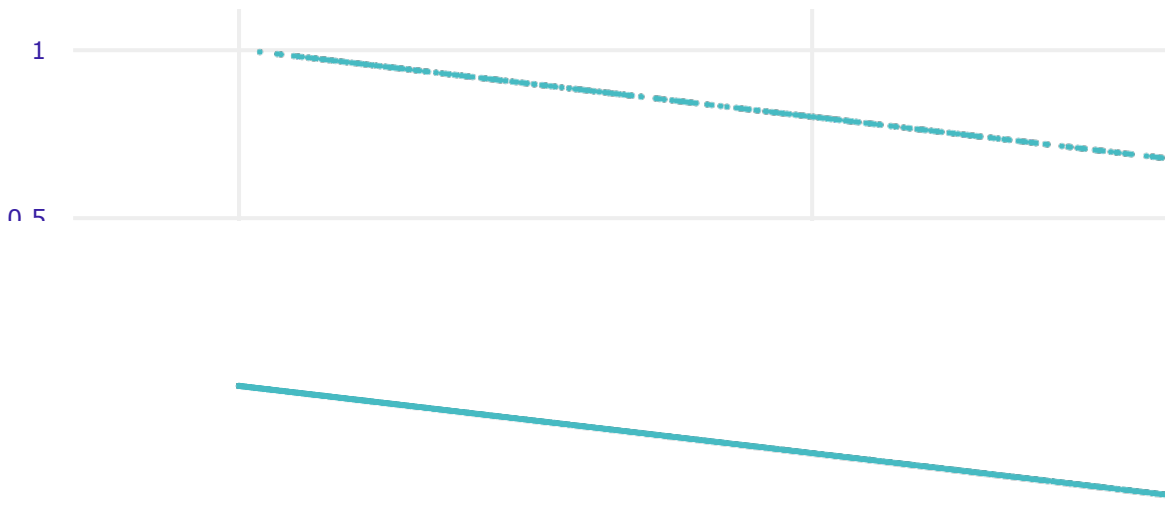
Residual Diagnostics



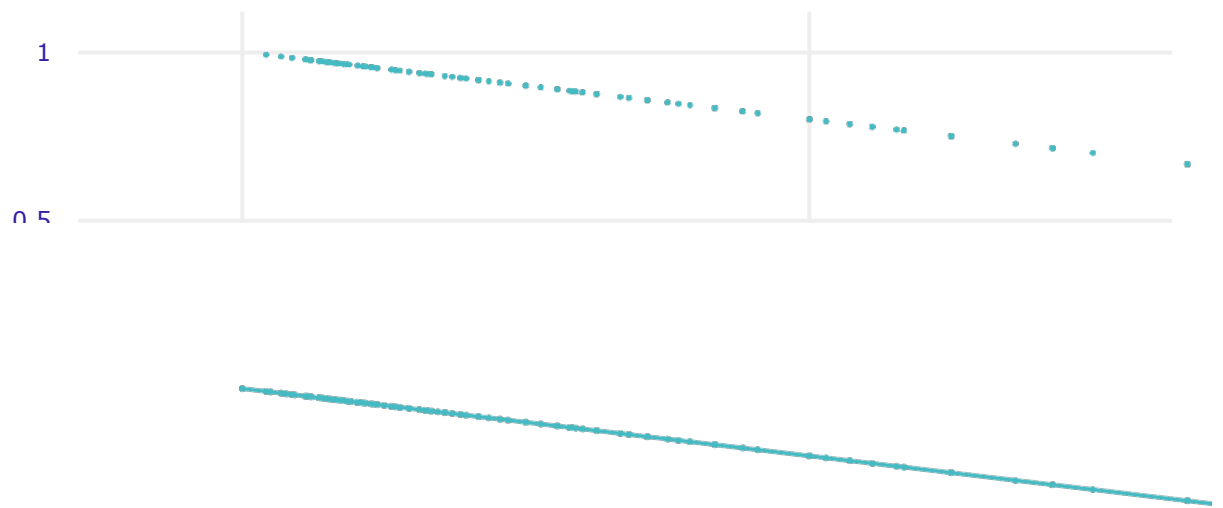
Residual Diagnostics



Residual Diagnostics

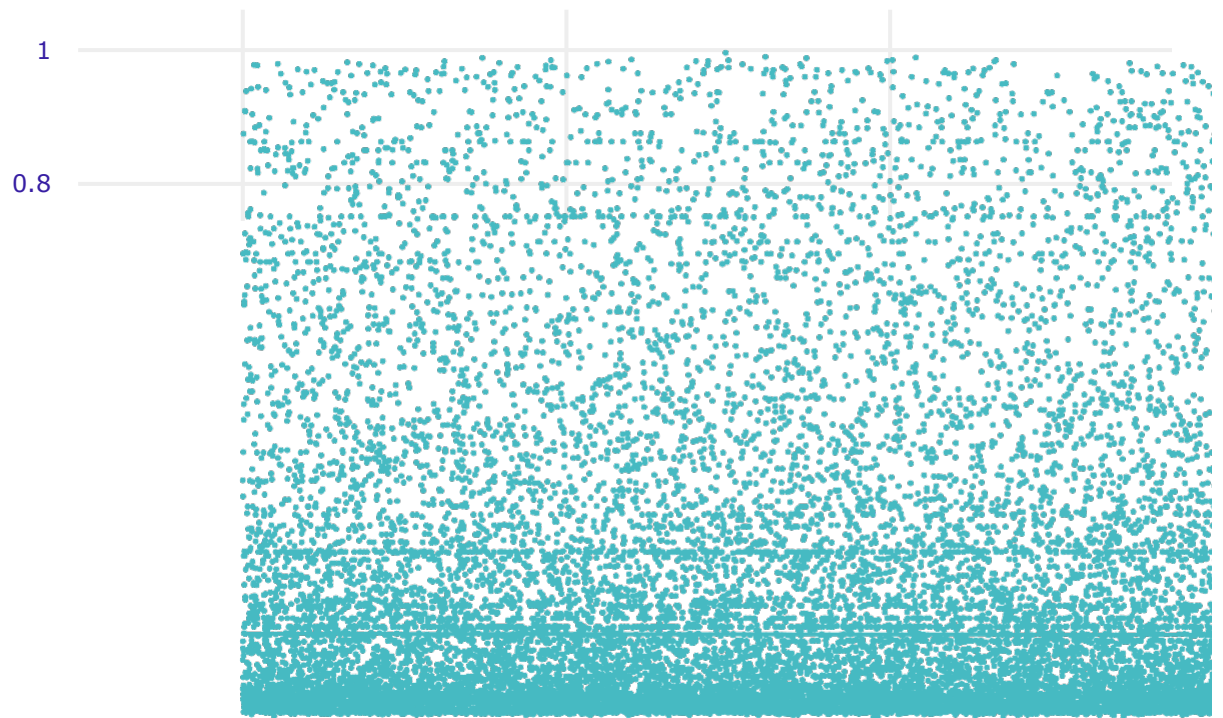


Residual Diagnostics

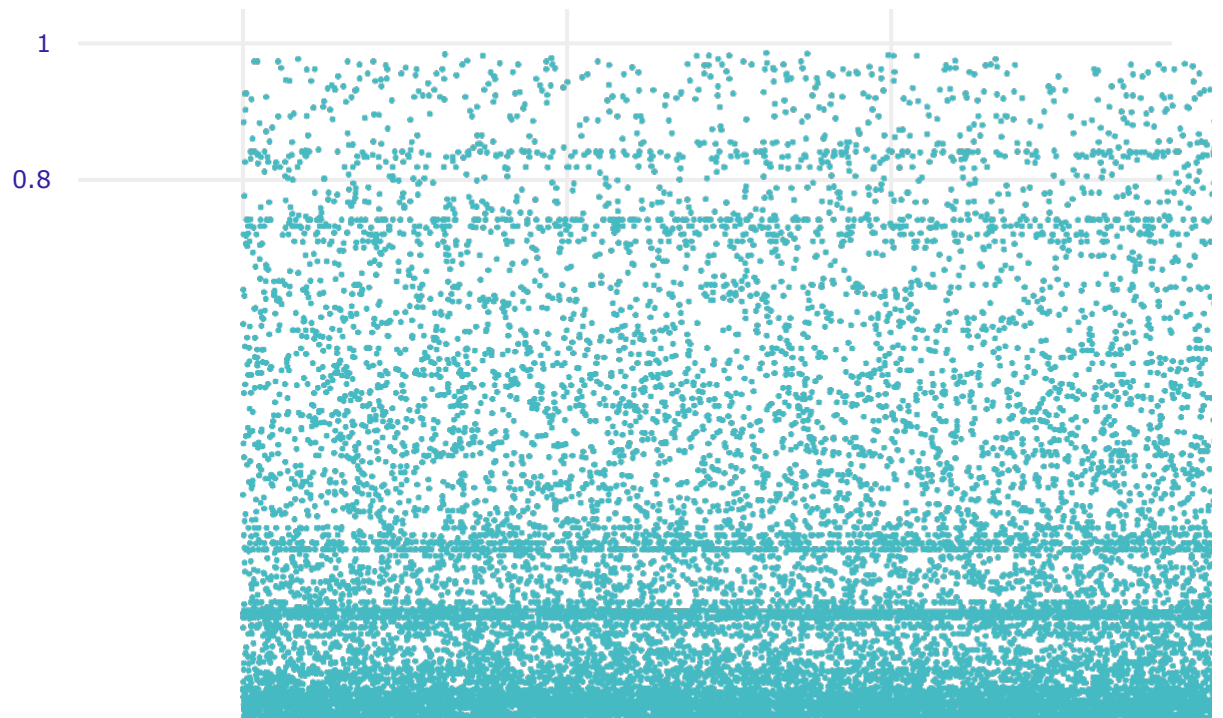


```
In [85]: # Plot Residual Diagnostics (Absolute)
for explainer in explainers.values():
    explainer.model_diagnostics().plot(variable = "ids", yvariable = "abs_residuals")
```

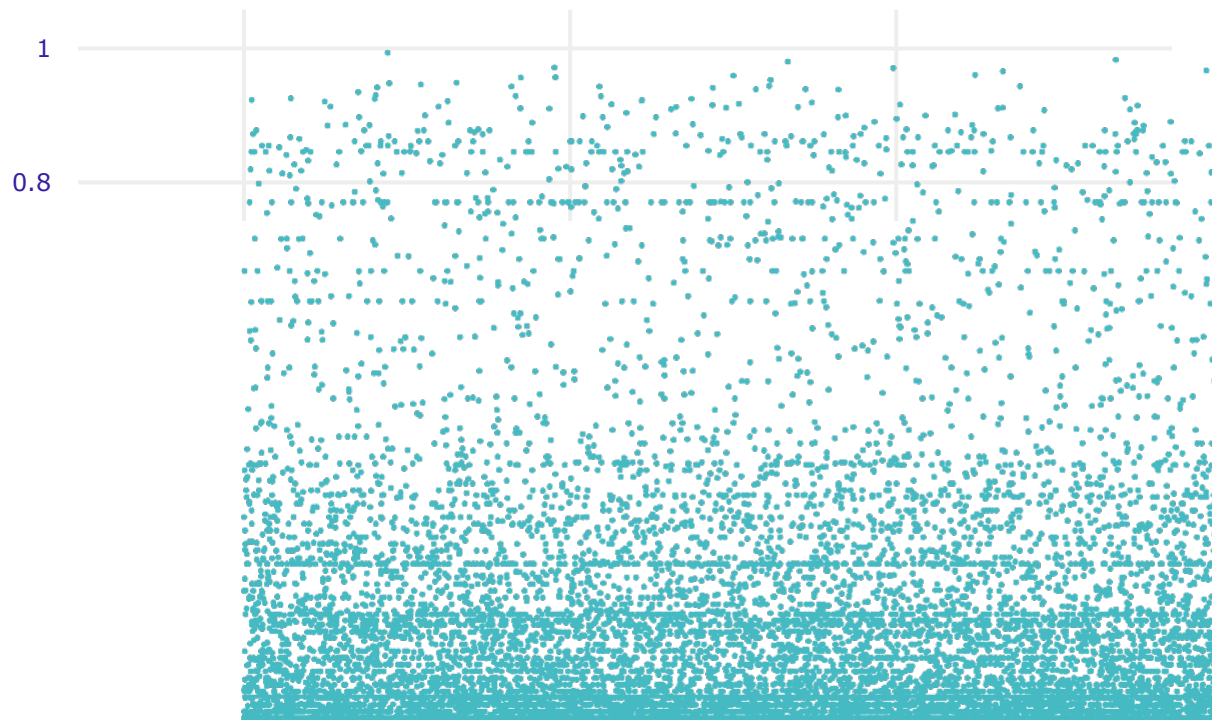
Residual Diagnostics



Residual Diagnostics



Residual Diagnostics



Residual Diagnostics



4.2. Global Explainability - Variable importances

```
In [86]: # Plot variable importances for selected models
for explainer in explainers.values():
    explainer.model_parts().plot()
```

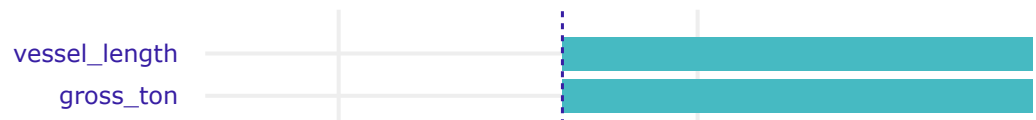
Variable Importance



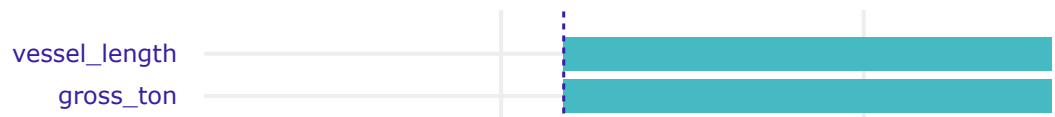
Variable Importance



Variable Importance



Variable Importance

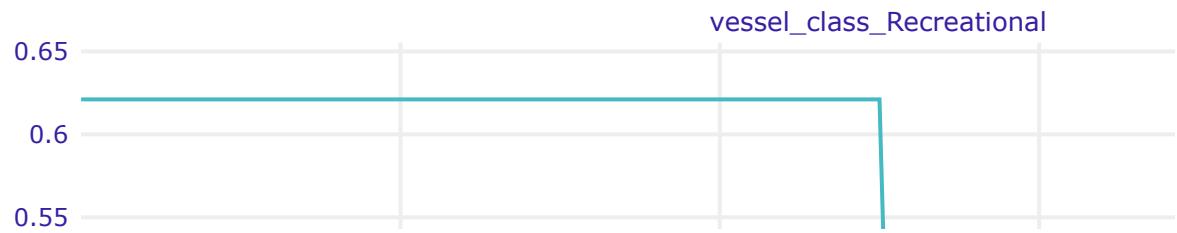


4.3. Partial Dependence Profile (PDP)

```
In [87]: # Plot PDP for selected models
for explainer in explainers.values():
    explainer.model_profile(variables = ["vessel_class_Recreational", "vessel_class_Towi
```

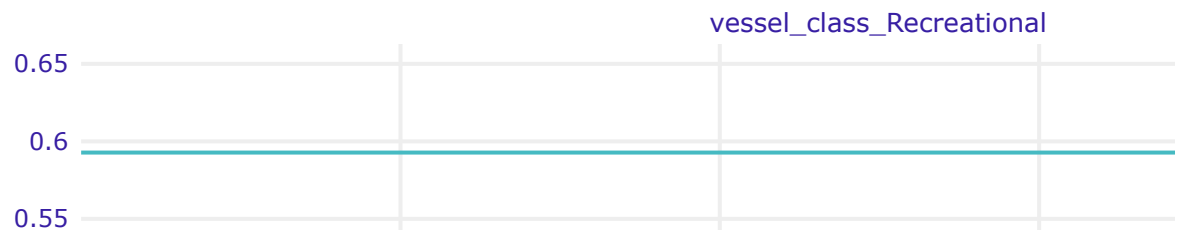
Calculating ceteris paribus: 100%|██████████| 2/2 [00:00<00:00, 21.72it/s]

Aggregated Profiles



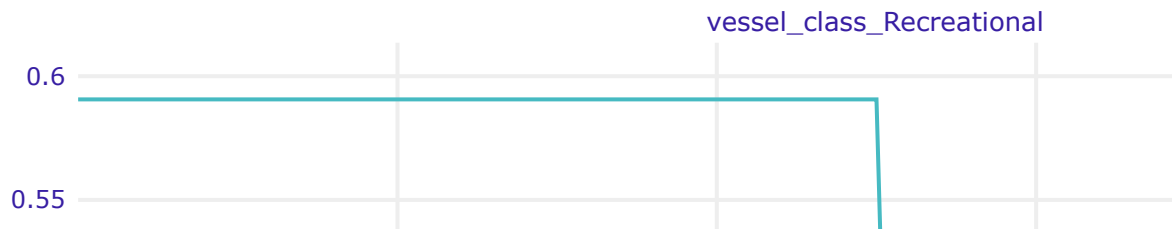
Calculating ceteris paribus: 100%|██████████| 2/2 [00:00<00:00, 105.10it/s]

Aggregated Profiles



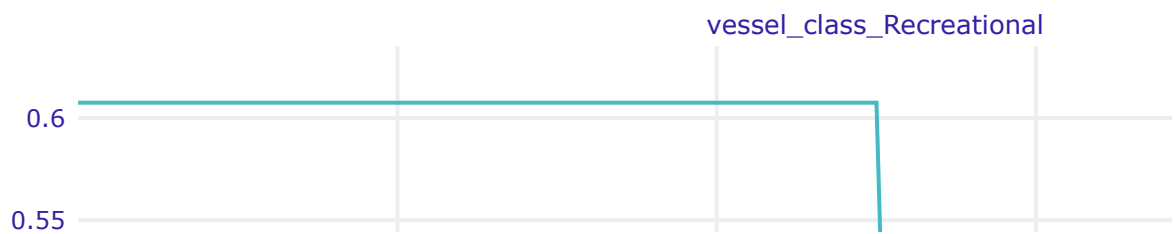
Calculating ceteris paribus: 100%|██████████| 2/2 [00:00<00:00, 6.91it/s]

Aggregated Profiles



Calculating ceteris paribus: 100% |  | 2/2 [00:00<00:00, 147.99it/s]

Aggregated Profiles



4.4. Break Down profiles

```
In [88]: # Select a sample observation
vessel_sample = X_train.sample(n=1, random_state=seed)

# Show this observation
vessel_sample
```

Out[88]:

	vessel_class_Barge	vessel_class_Bulk Carrier	vessel_class_Fishing Vessel	vessel_class_General Dry Cargo Ship	vessel_class_...
88779	1	0	0	0	

1 rows × 32 columns

```
In [89]: # Plot shapley values for selected observation
for explainer in explainers.values():
    explainer.predict_parts(new_observation = vessel_sample, type = "break_down").plot()
```

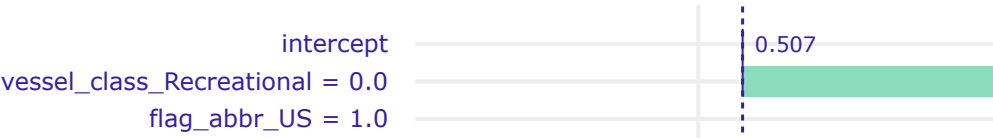
Break Down



Break Down



Break Down



Break Down



4.5. SHapley Additive exPlanations (SHAP)

```
In [90]: # Calculate predictions (time-consuming process)
if train_model_enabled :
    shaps = {}
    for model in explainers:
        shaps[model] = explainers[model].predict_parts(new_observation = vessel_sample,
        joblib.dump(shaps, models_folder + '/' + 'shaps.pkl')
else:
    shaps = joblib.load(models_folder + '/' + 'shaps.pkl')

# Plot shapley values
for model in explainers:
    shaps[model].plot()
```

Shapley Values



Shapley Values

vessel_class_Recreational = 0.0
vessel_length = 0.3626



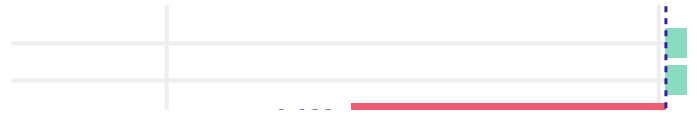
Shapley Values

vessel_length = 0.3626
build_year_new = 1.0



Shapley Values

```
vessel_length = 0.3626
gross_ton = -0.1901
```



4.6. Ceteris Paribus profiles

```
In [91]: # Select a sample observation
vessel_sample = X_train.sample()

# Plot shapley values for selected observation
for explainer in explainers.values():
    explainer.predict_profile(new_observation = vessel_sample).plot(variables = ["vessel",
```

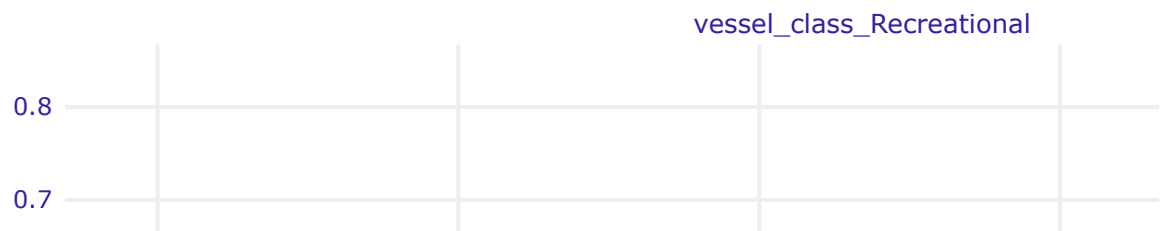
Calculating ceteris paribus: 100% | ██████████ | 32/32 [00:00<00:00, 586.63it/s]

Ceteris Paribus Profiles



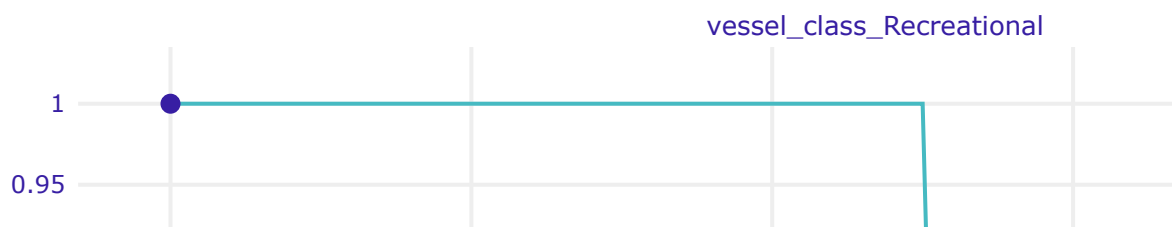
Calculating ceteris paribus: 100%|██████████| 32/32 [00:00<00:00, 282.97it/s]

Ceteris Paribus Profiles



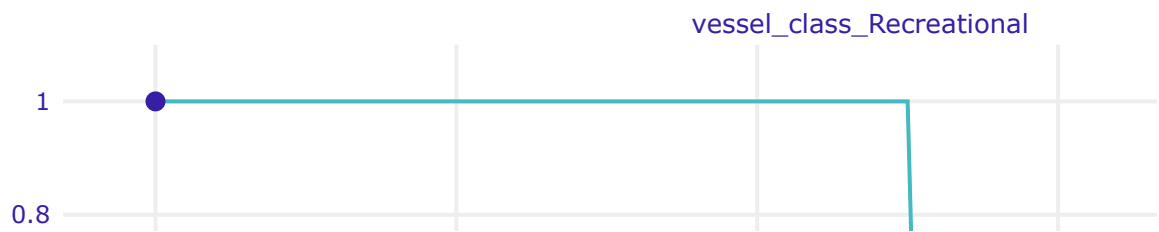
Calculating ceteris paribus: 100%|██████████| 32/32 [00:00<00:00, 297.59it/s]

Ceteris Paribus Profiles



Calculating ceteris paribus: 100%|██████████| 32/32 [00:00<00:00, 752.32it/s]

Ceteris Paribus Profiles



Session info

```
In [92]: # Python version
import sys
print(f'Python version: {sys.version}')

# Initiate variables for package versions
pkgs_dict = {}
version = []

# Loop importing __version__ for each package (because no whole packages are imported)
for pkg in ['pandas', 'numpy', 'sklearn', 'dalex']:
    exec(f"from {pkg} import __version__ as version")
    pkgs_dict[pkg] = version

# Show as table
pd.DataFrame(pkgs_dict, index=["Package version"])
```

Python version: 3.11.5 | packaged by Anaconda, Inc. | (main, Sep 11 2023, 13:26:23) [MSC v.1916 64 bit (AMD64)]

Out[92]:

	pandas	numpy	sklearn	dalex
Package version	2.2.2	1.25.2	1.4.2	1.7.0